Bazzite Unified Home Server + Gaming Station Guide

SSH-first · Zero New Open Ports · Headless by Default

bazzite-deck | Docker | firewalld | Fail2ban | Nginx Proxy Manager | Nextcloud AIO | DuckDNS | krdp | gamescope-session-steam | SELinux | systemd-run

A single, self-contained guide that builds a headless home server with two on-demand graphical sessions — KDE Plasma and Gaming Mode — from a single `bazzite-deck` installation. All phases are additive and ordered. No rebase required. Written for beginners who want to understand exactly what every command does and why.

CONTENTS — Table of Contents

Phase 0	OS Installation (bazzite-deck from ISO)
Phase 1	First Boot: SSH Bootstrap & Headless Server Configuration
Phase 2	Router & Network Preparation
Phase 3	System Package Installation (Docker · Fail2ban · SELinux tools)
Phase 4	Ironclad Security (SELinux Port · SSH Keys · firewalld · Fail2ban)
Phase 5	Docker Initialisation
Phase 6	Data Drive Setup
Phase 7	Deploying Services (DuckDNS · NPM · Nextcloud AIO)
Phase 8	Session Management Scripts (start-kde · start-gaming · stop-session)
Phase 9	krdp Remote Desktop
Phase 10	Link & Initialise (NPM config · Nextcloud setup)
Phase 11	Testing & Backups
App. A	Troubleshooting
App. B	Normal Workflows (Quick Reference)
App. C	Optional: AMD GPU Hardware Acceleration for Nextcloud
App. D	OS Updates & Rollbacks

OVERVIEW — What You Get & Architecture

What is this setup? This guide turns a single desktop PC into two things at once: a 24/7 home server running self-hosted cloud software, and an on-demand gaming station. The trick is that the machine normally sits at a blank, idle login screen (consuming almost no resources). You trigger either KDE Desktop or Steam Gaming Mode remotely over SSH whenever you want them — then the machine returns to idle automatically. All the server software (Nextcloud, reverse proxy, DNS updater) runs in isolated Docker containers in the background, completely independent of whether any graphical session is active.

Component	What You Get
Headless server default	Boots to an idle SDDM login screen. Docker containers run as systemd services. SSH always available on port 22022.
KDE Wayland session	Full KDE Plasma on the physical display, on demand via SSH. Enables krdp remote desktop.
Gaming Mode session	Steam Deck-style gamescope session. Direct GPU scanout, controller-native. Refused if another user session is active.
krdp remote desktop	KDE's native Wayland RDP server. Tunnelled exclusively through SSH — port 3389 blocked by firewalld.
Nextcloud AIO	Self-hosted cloud: files, calendar, contacts. Behind Nginx Proxy Manager with Let's Encrypt via DuckDNS.
SSH-first design	Every operation is a single SSH command. No new ports beyond 22022 are ever opened to the public internet.

Architecture Explained

bazzite-deck ships both gamescope Gaming Mode and KDE Plasma in one image. On first boot the machine enters Gaming Mode once (expected). Phase 1 applies a headless SDDM override so every subsequent boot lands at an idle login screen. Docker containers start at boot via systemd, completely independently of any graphical session.

Key concept: systemd is the init system — the first process that starts when Linux boots. It starts and manages every service, including SSH, Docker, and the display manager. Think of it as Linux's task manager and service controller combined. Commands that start with `systemctl` talk to systemd.

Key concept: Docker runs software inside isolated "containers" — like lightweight virtual machines, but faster and sharing the host kernel. Each service (Nextcloud, NPM, DuckDNS) runs in its own container with its own files and network, isolated from the host OS. Even if a container is compromised, it cannot easily affect the host or other containers.

Recommended Minimum Hardware

Component	Minimum / Recommended
CPU	4-core x86_64 minimum · 6-core or better recommended
RAM	8 GB DDR4 minimum · 16 GB+ recommended (Fulltextsearch requires 16 GB+)
OS Drive	120 GB NVMe SSD minimum · 500 GB+ NVMe recommended
Data Drive	Any size HDD or SSD · Dedicated, mounted at /srv/nextcloud-data
GPU	AMD recommended (plug-and-play) · NVIDIA: beta support for Gaming Mode only
Network	Gigabit Ethernet · Wi-Fi strongly discouraged for servers

Action Flow

Action	Flow
Launch KDE	SSH → `sudo start-kde.sh` → writes one-shot SDDM autologin → restarts SDDM → KDE Wayland appears → systemd-run timer restores headless config 10 s later.
Remote desktop	KDE must be running. SSH tunnel maps local port 13389 to server port 3389. Connect any RDP client to `localhost:13389`. krdp captures compositor output — monitors can be off.
Gaming Mode	SSH → `sudo start-gaming.sh` → checks seat0 for active user session → writes one-shot SDDM autologin → gamescope session appears. Steam Power menu exits to idle SDDM.

Critical Pitfalls & How This Guide Avoids Them

#	What Goes Wrong & The Fix
1	bazzite-deck boots to Gaming Mode on first install — enter Desktop Mode, enable SSH, apply the headless SDDM override. All in one physical session.
2	SSH is not enabled by default on Bazzite — the very first action in Phase 1 Step 2 enables sshd.
3	SDDM reads config once per start — start scripts write a one-shot autologin config, restart SDDM, then a systemd-run transient timer restores the headless config 10 s later (immune to session cgroup cleanup).
4	krdp listens on ALL interfaces — firewalld blocks port 3389. Never open port 3389 in firewalld.
5	krdp requires an active KDE session — always run `start-kde.sh` first.
6	Sudoers entries require absolute paths and root-owned scripts — this guide uses `/usr/local/bin/` with `root:root` ownership.
7	SELinux blocks sshd on non-standard ports — `semanage port` must label 22022 BEFORE restarting sshd.
8	Package installation must precede security hardening — all packages are installed in Phase 3 before any hardening begins.
9	Auto-suspend silently kills all containers — Phase 1 Step 3 permanently masks all suspend targets.
10	Docker bypasses firewalld for published ports (80, 443) via direct nftables injection. Ports 80 and 443 ARE open to the internet even though `firewall-cmd --list-all` does not show them. Never add firewalld rules for 80, 443, or 3389.
11	Missing `nofail` on the secondary drive causes systemd to drop into emergency mode on failure, disabling SSH entirely. The fstab entry in Phase 6 always uses `defaults,nofail`.
12	`KillUserProcesses=yes` in systemd-logind kills background subshells when the SSH session ends. The start scripts use `systemd-run` to schedule headless restoration as a system-level transient service — immune to session cgroup cleanup.
13	Blind mount on data drive failure: with `nofail`, if the drive is absent `/srv/nextcloud-data` becomes an empty folder on the OS drive. Fix: `sudo chattr +i /srv/nextcloud-data` makes the directory immutable. Always apply this flag while the drive is **unmounted** — if applied while mounted, it sets the flag on the data drive itself rather than the OS-side directory, which denies Nextcloud write access to its own data. The kernel automatically overrides the OS-side immutable flag when the real ext4 partition mounts over the directory.
14	The `gamescope-session-plus.desktop` session file was renamed to `gamescope-session-steam.desktop` in current Bazzite builds. The `start-gaming.sh` script auto-lists available sessions if the configured name is not found.

PHASE 0 — OS Installation

bazzite-deck from ISO · Done physically at the machine

What is an ISO? An ISO file is a complete disc image of an operating system installer. You copy it to a USB drive, boot from that USB drive, and the installer sets up the OS on your machine's internal drive. Think of it as a digital installation disc.

Why bazzite-deck specifically? Bazzite has two main variants: the standard desktop variant, and `bazzite-deck` — the HTPC/living-room variant. The `-deck` suffix means it ships both *gamescope-session-steam* (Steam Deck-style Gaming Mode) and KDE Plasma in a single signed image. Starting with `bazzite-deck` eliminates a separate rebase step, keeps Gaming Mode as part of the verified OS image (not a layered add-on), and ensures every OS update maintains Gaming Mode infrastructure.

Step 1 · Download the correct ISO

Go to `bazzite.gg` and use the Image Picker tool. Select: **Hardware = Desktop/HTPC**, **GPU = AMD/Intel or NVIDIA**, **Desktop = KDE Plasma**, and enable **Steam Gaming Mode** (this selects the `bazzite-deck` variant). Download the ISO and flash it to a USB drive using **Fedora Media Writer** or **Balena Etcher**.

Step 2 · Install Bazzite

Boot from the USB drive and follow the **Anaconda installer** (Fedora's graphical installer). Set your username, password, hostname (under 20 characters, no spaces), timezone, and keyboard layout. Install to your OS drive. A network connection is not required during installation.

> **Dual Boot Warning:** If you have a Windows drive in the same machine, physically disconnect it before installing Bazzite to prevent accidental data loss or bootloader modification.

> **Why keep the hostname under 20 characters?** The hostname identifies your machine on the network and appears in SSH prompts and logs. Very long hostnames can cause issues with some older network tools and SDDM session management. Keep it short and descriptive, e.g., `homeserver` or `bazzite-server`.

Step 3 · Note your GPU type for later reference

AMD and Intel GPUs are fully supported in Gaming Mode. NVIDIA support in Gaming Mode is available but described as beta — expect potential issues with direct GPU scanout. Appendix C (AMD GPU hardware acceleration for Nextcloud) requires an AMD or Intel GPU.

PHASE 1 — First Boot: SSH Bootstrap & Headless Server Configuration

After the first reboot following installation, bazzite-deck boots directly into Gaming Mode (Steam Big Picture). This is expected and intentional. Phase 1 converts the machine into a headless server during this very first boot.

> **Do ALL steps in this phase in sequence before rebooting.** The headless SDDM override in Step 4 must be in place before your next reboot. If you reboot before completing Step 4, the machine will boot to Gaming Mode again — simply repeat this phase.

Step 1 · Enter KDE Desktop Mode from Gaming Mode

You will see Steam Big Picture on your TV or monitor. Navigate to the **Power menu** (bottom-right power icon) and select **Switch to Desktop**. KDE Plasma will launch on the physical display.

> **What is KDE Plasma?** KDE Plasma is a full desktop environment — the graphical workspace with a taskbar, file manager, application menu, and system settings. On bazzite-deck, it coexists with Gaming Mode, and you can switch between them. For the server setup, we need KDE's terminal (Konsole) to run commands with keyboard input.

Step 2 · Open Konsole and enable SSH

Open **Konsole** (the terminal emulator) from the application menu. SSH is not enabled by default on Bazzite.

> **What is SSH?** SSH (Secure Shell) is an encrypted protocol for remotely accessing a computer over a network. Once enabled, you can type commands on your server from your personal computer without ever touching the server's keyboard or monitor again. The *SSH daemon* (`sshd`) is the background service on the server that listens for incoming SSH connections.

```
# Enable and start the SSH daemon.
# — sudo: run the following command as root (the system superuser).
#   Most system administration commands require root privileges.
# — systemctl: the command-line tool for managing systemd services.
# — enable: register this service to start automatically on every future boot.
#   It creates a symbolic link in systemd's startup sequence.
# — --now: also start the service immediately, right now, without waiting for a reboot.
# — sshd: the OpenSSH server daemon.
sudo systemctl enable --now sshd

# Find your active network interface and local IP address.
# — ip route get 1.1.1.1: ask the kernel's routing table which network
#   interface it would use to reach 1.1.1.1 (Cloudflare's public DNS server).
#   We don't actually send traffic there — we just use it to force the kernel
#   to reveal the active interface and your machine's local IP address.
# Look for: dev <interface> src <your-IP>
# Example output: ... dev enp3s0 src 192.168.1.50 ...
ip route get 1.1.1.1

# Alternative if your LAN has no internet access at setup time:
# — ip a: list all network interfaces and their IP addresses.
#   Look for the "inet 192.168.x.x" line under your ethernet interface.
ip a

# Write down the IP address — you will use it for every SSH connection
# throughout the rest of this guide.
```

> **Interface names:** Modern Linux uses predictable interface names like `enp3s0`, `eno1`, `enp4s0` instead of the old `eth0`. The name encodes the bus and slot position of the network card, making it stable across reboots. `ip route get 1.1.1.1` reliably identifies your active interface regardless of its name.

Step 3 · Disable auto-suspend (CRITICAL for servers)

Bazzite may suspend the machine after inactivity, silently killing all Docker containers. A server must stay on 24/7. Permanently disable all suspend modes using two independent methods.

What is "masking" a systemd target? `systemctl disable` prevents a service from starting automatically but still allows it to be started manually. `systemctl mask` is stronger — it replaces the service file with a symlink pointing to `/dev/null`, making it impossible to start by any means until unmasked. For a server, we want an absolute guarantee that suspend can never activate, even if some other software tries to trigger it.

Step A — KDE System Settings (UI-level prevention): System Settings → Power Management → Energy Saving → On AC Power → set "Suspend session" to **Never**. Click Apply.

Step B — Mask suspend targets at the OS level (the authoritative method):

```
# Permanently prevent all forms of suspend — even when no desktop is active.
# These five targets cover every suspend variant Linux supports:
# — sleep.target:                  generic sleep (catches umbrella sleep requests)
# — suspend.target:                standard suspend-to-RAM (S3 sleep)
# — hibernate.target:              write RAM to disk, then power off (S4)
# — hybrid-sleep.target:           suspend-to-RAM + writes hibernate image
# — suspend-then-hibernate.target: suspends first, then hibernates after a delay
sudo systemctl mask sleep.target suspend.target \
  hibernate.target hybrid-sleep.target suspend-then-hibernate.target

# Verify all five are masked.
# — grep -E "Loaded:|●": filter output to show only the Loaded: status line
#   and the unit's header line. "masked" must appear next to all five.
systemctl status sleep.target suspend.target \
  hibernate.target hybrid-sleep.target suspend-then-hibernate.target \
  | grep -E "Loaded:|●"
# Expected: every entry shows "masked" in the Loaded: line.
```

Step 4 · Apply the headless SDDM override (THE MOST CRITICAL STEP)

What is SDDM? SDDM (Simple Desktop Display Manager) is the login screen manager. It starts at boot, shows a login screen, and launches a graphical session when a user logs in. On bazzite-deck, it is pre-configured to autologin into Gaming Mode on every boot — which is fine for a gaming console, but we need to change this for a server. We want SDDM to start but show an idle blank login screen, with no automatic login into any session.

What is a drop-in config file? Instead of editing a single monolithic config file (which could be overwritten by OS updates), modern Linux software supports "drop-in" directories. SDDM reads config files from `/etc/sddm.conf.d/` in alphabetical filename order, merging all values together. Files processed later override earlier ones. By naming our file `zzz-headless-server.conf`, the `zzz-` prefix guarantees it is processed absolutely last — after all system defaults and bazzite's pre-existing configs — so our empty `User=` and `Session=` fields win unconditionally.

What is a heredoc (`<< 'EOF'`)? A heredoc is a way to write multi-line text directly in a shell command, without needing a text editor. Everything between the two `EOF` markers is treated as file content. The single quotes around `'EOF'` prevent the shell from expanding any `$variables` or `$(commands)` inside — the text is written exactly as-is. The `tee` command reads from standard input and writes to a file while also printing to the terminal.

```
# Create the config directory if it does not exist.
# — mkdir -p: create the directory and any missing parent directories.
#   Without -p, this command would fail if /etc/sddm.conf.d already exists.
sudo mkdir -p /etc/sddm.conf.d

# Write the headless override file.
# — sudo tee FILE: tee reads from stdin and writes to FILE as root.
#   We need root to write into /etc/, which your user cannot write to directly.
# — > /dev/null: discard tee's stdout copy (the terminal echo). We already see
#   the content inside the heredoc, so suppressing the duplicate is cleaner.
# — << 'EOF': start a heredoc. Everything until the closing EOF line is the
#   file content. Single quotes prevent any variable expansion inside.
# The resulting file tells SDDM: autologin User=(empty) — meaning: no autologin.
sudo tee /etc/sddm.conf.d/zzz-headless-server.conf > /dev/null << 'EOF'
[Autologin]
User=
Session=
Relogin=false
EOF

# Verify it was written correctly — the file must show the exact content above:
cat /etc/sddm.conf.d/zzz-headless-server.conf

# Apply the new config now without rebooting.
# — systemctl restart sddm: stop and restart the SDDM process.
#   SDDM re-reads all its config files on start, picking up our zzz- override.
# Your KDE session may end — that is expected and intentional. SSH still works.
sudo systemctl restart sddm
```

Step 5 · Verify the headless config survives a reboot

```
# Reboot the server:
# — systemctl reboot: a clean, orderly reboot. Stops all services gracefully
#   before restarting. Faster and safer than the physical power button.
sudo systemctl reboot

# Wait approximately 60 seconds for the machine to boot.
# The physical display should show the SDDM login screen — NOT Gaming Mode.

# Connect from your personal computer over SSH on the default port 22.
# (Port 22 is used here because we have not hardened SSH to port 22022 yet.)
# — ssh username@IP: connect as "username" to the server at this IP address.
#   Replace with your actual Bazzite username and server IP from Step 2.
ssh username@192.168.1.50

# Confirm no graphical session is running.
# — loginctl list-sessions: list all active sessions tracked by systemd-logind.
#   A "session" here is a login instance — SSH sessions, graphical sessions, etc.
#   seat0 is the physical display hardware. An entry on seat0 means a graphical
#   session is active on your TV/monitor.
loginctl list-sessions
# Expected: empty output, or only your SSH session with no seat0 column entry.
```

If you cannot SSH in on port 22: Boot back to Desktop Mode via Gaming Mode Power menu → Switch to Desktop, re-run Step 2, then Step 5.

PHASE 2 — Router & Network Preparation

Done on your personal computer · Requires access to your router's admin panel

Why configure the router? Your router is the gateway between your home network and the internet. Three things need router configuration: (1) giving the server a permanent local IP address so it never changes, (2) allowing specific internet traffic to reach your server (port forwarding), and (3) setting up a domain name so you can reach your server from anywhere by name rather than by a changing IP address.

Step 1 · Assign a static local IP (DHCP reservation)

DHCP and why we need a static IP: By default, your router hands out IP addresses automatically (DHCP — Dynamic Host Configuration Protocol). These addresses can change at any time. If your server's IP changes, port forwarding breaks, SSH connections fail, and local DNS entries stop working. A *DHCP reservation* (also called "Static Lease" or "Address Reservation") tells the router: "always give this specific network card (identified by its MAC address) this specific IP address." The server still gets its IP from DHCP automatically — the address is just guaranteed to never change.

In your router's admin panel (typically at `192.168.1.1` or `192.168.0.1`): find the DHCP section, add a reservation for your server's MAC address, and assign the IP you noted in Phase 1 Step 2.

Step 2 · Set up port forwarding

What is port forwarding? Your router has one public IP address (from your ISP) and shares it among all devices on your home network via NAT (Network Address Translation). Port forwarding creates a permanent rule: "when traffic arrives from the internet on port X, send it to the server at 192.168.1.50." Without this, internet traffic has no way to reach your server. We open only the specific ports that need to be publicly reachable.

Port / Protocol	Purpose
80 TCP	Required by Let's Encrypt for SSL certificate issuance (HTTP-01 challenge). Let's Encrypt's servers make an HTTP request to your domain to verify you control it before issuing a certificate.
443 TCP	Your Nextcloud's public HTTPS entry point. All user traffic to Nextcloud uses this port.
22022 TCP	SSH — our custom hardened port (configured in Phase 4).
3478 TCP+UDP	Required ONLY if you plan to enable Nextcloud Talk (video calls) later.

NEVER forward port 3389 (RDP). krdp is only accessible via SSH tunnel. Port 3389 must never be forwarded or opened in firewalld. See Phase 9.

Ports 80 and 443 — security model clarification: These ports are open to the internet through your router. Docker manages them by injecting rules directly into the kernel's packet filtering (nftables) — bypassing the firewall tool (firewalld). This means `firewall-cmd --list-all` will NOT show ports 80 or 443 as open, even though the server is actively accepting global HTTP/S traffic. This is expected behaviour. Nginx Proxy Manager (NPM) handles all routing and enforces HTTPS; no internal service is directly reachable from the internet through NPM without an explicit proxy host entry.

Step 3 · Create your DuckDNS account

> **What is DuckDNS and why do we need it?** Your home internet connection has a public IP address assigned by your ISP. This IP can change at any time (dynamic IP). DuckDNS is a free dynamic DNS service: it gives you a permanent domain name (e.g., `yourname.duckdns.org`) and provides a tiny updater program you run on your server. Whenever your public IP changes, the updater notifies DuckDNS and the domain name is updated within 60 seconds. This means you can always reach your server at the same URL — even if the underlying IP changes. The token is a secret key that proves you own your DuckDNS subdomain.

Go to `duckdns.org` and sign in with Google or GitHub. Create a free subdomain (e.g., `yourname.duckdns.org`) and copy your **Token**. Note both values — you will need them in Phase 7.

PHASE 3 — System Package Installation

Docker · Fail2ban · SELinux tools · Runs on port 22 before SSH hardening

> **What is rpm-ostree and the immutable OS?** Bazzite is built on Fedora Atomic — an "immutable" operating system. The root filesystem is read-only (locked). This means OS updates are atomic: a new OS image is downloaded in the background and applied in full on the next reboot. If an update breaks something, one command rolls back to the previous image. The downside is that you cannot install packages the traditional way (`dnf install`). Instead, `rpm-ostree install` "layers" packages on top of the immutable base — they're added to a separate writable layer that persists across updates. A reboot is required to activate any newly layered packages.

Step 1 · OS updates — no action needed

Bazzite updates arrive as complete new images staged in the background, activated atomically on the next reboot. If an OS update ever causes an issue, one command returns to the previous working state:

```
# Roll back to the previous OS image and reboot into it.
# rpm-ostree keeps your last two deployments, so rollback is always possible.
rpm-ostree rollback && systemctl reboot
```

Step 2 · Add the Docker repository

> **Why Docker CE and not Podman?** Bazzite comes with Podman pre-installed. Podman is a rootless container engine compatible with Docker commands, but it has limitations. Nextcloud All-in-One's mastercontainer manages multiple child containers by connecting directly to the Docker API socket. This orchestration logic requires a native Docker API; Podman's Docker compatibility layer is insufficient for AIO's needs. We install Docker CE (Community Edition) — the upstream, full-featured Docker engine.

```
# Download the Docker CE repository definition for Fedora and save it to
# the system's package repository configuration directory.
# — curl: a command-line tool for downloading files from URLs.
# — -f: fail silently on server errors (don't save an HTML error page as the file).
# — -s: silent mode — don't show progress. We add -S to still show errors if any.
# — -S: show errors even when -s suppresses output. Combined: -fsSL.
# — -L: follow redirects (if the URL redirects to a new location, follow it).
# — -o FILE: save the downloaded content to FILE instead of printing to terminal.
sudo curl -fsSL https://download.docker.com/linux/fedora/docker-ce.repo \
  -o /etc/yum.repos.d/docker-ce.repo
```

Step 3 · Install all required packages (one reboot required)

> **What is the podman-docker shim?** Bazzite may have a package called `podman-docker` installed. This shim provides a `/usr/bin/docker` symlink that points to Podman, making Podman respond to `docker` commands. If we install Docker CE while this shim is present, both packages try to own `/usr/bin/docker` and conflict. We detect whether the shim is installed and remove it atomically in the same operation as installing Docker CE, preventing any conflict.

```
# Check if the podman-docker shim is installed.
# — rpm -q: query the RPM package database.
# — >/dev/null 2>&1: discard all output (stdout to /dev/null, stderr to stdout).
#   We only care about the exit code (0 = installed, 1 = not installed).
if rpm -q podman-docker >/dev/null 2>&1; then
  # Shim IS present — remove it and install Docker CE in one atomic operation.
  # rpm-ostree override remove: remove a package that is part of the base image.
  # --install: simultaneously install additional packages.
  # This is atomic: either all changes apply together, or none do.
  rpm-ostree override remove podman-docker \
    --install docker-ce \
    --install docker-ce-cli \
    --install containerd.io \
    --install docker-buildx-plugin \
    --install docker-compose-plugin \
    --install fail2ban \
    --install fail2ban-firewalld \
    --install policycoreutils-python-utils
else
  # Shim is NOT present — straightforward install.
  rpm-ostree install \
    docker-ce \
    docker-ce-cli \
    containerd.io \
    docker-buildx-plugin \
    docker-compose-plugin \
    fail2ban \
    fail2ban-firewalld \
    policycoreutils-python-utils
fi

# Reboot to activate the newly layered packages.
systemctl reboot
```

Package explanations:

- `docker-ce` : The Docker Engine — the core daemon that runs containers.
- `docker-ce-cli` : The `docker` command-line tool you type commands into.
- `containerd.io` : The container runtime that Docker uses under the hood to actually start and stop containers.
- `docker-buildx-plugin` : Extended build capabilities for Docker images. Included for completeness.
- `docker-compose-plugin` : Enables the `docker compose` command for managing multi-container applications defined in YAML files.
- `fail2ban` : Monitors log files and automatically bans IP addresses that show signs of brute-force attacks.
- `fail2ban-firewalld` : The Fail2ban action module that integrates with firewalld (instead of older iptables).
- `policycoreutils-python-utils` : Provides the `semanage` command, which is required in Phase 4 to manage SELinux port labels.

Your SSH session will disconnect. `systemctl reboot` terminates the SSH connection immediately with "broken pipe." This is expected — the machine is rebooting. Wait approximately 60 seconds, then reconnect: `ssh username@192.168.1.50`

Do not use `ujust install-docker`. Bazzite's built-in shortcut has a confirmed bug (GitHub #2130): the Docker service fails to start after install and docker-compose is not included. The rpm-ostree method above is the correct path.

PHASE 4 — Ironclad Security

SELinux Port · SSH Keys · firewalld · Fail2ban · Steps 1, 3–5 on server via SSH port 22 · Step 2 on personal computer

What is SELinux? SELinux (Security-Enhanced Linux) is a mandatory access control system built into the Linux kernel. Unlike traditional permissions (which only control who owns a file), SELinux attaches security "labels" to every file, process, and network port. A process can only access resources with compatible labels. It runs in *enforcing mode* on Bazzite — violations are blocked, not just logged. This is why certain operations require explicit SELinux configuration before they will work.

Step 1 · Label port 22022 in the SELinux policy (Server — MUST BE FIRST)

SELinux controls which ports the SSH daemon is permitted to bind to. Only port 22 has the `ssh_port_t` label by default. If we restart sshd on port 22022 without adding this label first, SELinux will silently block the bind syscall — sshd will appear to start successfully but will not actually listen on any port, locking you out permanently.

```
# Add port 22022 to SELinux's list of allowed SSH ports.
# — semanage port: manage SELinux port label assignments.
# — -a: add a new rule (as opposed to -m modify or -d delete).
# — -t ssh_port_t: assign the SSH port type label. This is the label that
#   the sshd process is permitted to bind to, as defined in SELinux policy.
# — -p tcp: this label applies to the TCP protocol (not UDP).
# — 22022: the port number to label.
sudo semanage port -a -t ssh_port_t -p tcp 22022

# Verify the label was added.
# — semanage port -l: list all SELinux port context assignments.
# — | grep ssh: filter the output to show only SSH-related lines.
# The output must include both 22 and 22022 on the ssh_port_t line.
sudo semanage port -l | grep ssh
# Expected: ssh_port_t    tcp    22022, 22
```

Do not skip or reorder this step. sshd will fail to bind if you restart it on port 22022 before this SELinux label is applied. Recovery requires a physical display and keyboard to enter emergency mode.

Step 2 · Generate and copy SSH keys (Personal Computer)

What are SSH keys and why are they more secure than passwords? SSH key authentication uses cryptography instead of passwords. You generate a *key pair*: a private key (stays on your computer, never shared) and a public key (copied to the server). To log in, your SSH client proves it has the private key by performing a mathematical operation that can only be done with that key. The server verifies this using the public key. An attacker who intercepts the network traffic or brute-forces the server gets nothing useful — there is no password to guess or intercept. After this phase, password authentication is permanently disabled.

Linux / macOS / WSL:

```
# Generate a new ed25519 key pair.
# — ssh-keygen: the key generation tool.
# — -t ed25519: use the Ed25519 algorithm — a modern elliptic-curve algorithm
#   that is faster, smaller, and more secure than the older RSA algorithm.
#   Skip this line if you already have an SSH key pair (~/.ssh/id_ed25519).
ssh-keygen -t ed25519

# Copy your public key to the server's authorized_keys file.
# — ssh-copy-id: a helper tool that reads your public key, SSHes to the server
#   with your password, and appends the key to ~/.ssh/authorized_keys.
#   After this, SSH will use your key instead of your password.
# — username@192.168.1.50: replace with your actual username and server IP.
ssh-copy-id username@192.168.1.50

# Test that key-based login works before we disable passwords.
ssh username@192.168.1.50
# Expected: you log in without being prompted for a password.
```

Windows (PowerShell):

```
# Generate a key pair:
ssh-keygen -t ed25519

# Copy the public key using scp (binary-safe file copy over SSH).
# We use scp here instead of PowerShell's type or cat because PowerShell's
# built-in file reading adds UTF-16LE encoding and Windows CRLF line endings,
# which silently corrupt the authorized_keys file on Linux.
# scp transfers the file as raw bytes, preserving exact content.
scp $env:USERPROFILE\.ssh\id_ed25519.pub username@192.168.1.50:~/id_ed25519.pub

# Append it to authorized_keys on the server and clean up.
# — mkdir -p ~/.ssh: create the .ssh directory if it doesn't exist.
# — cat ~/id_ed25519.pub >> ~/.ssh/authorized_keys: append (not overwrite) the key.
# — rm ~/id_ed25519.pub: clean up the temporary copy.
ssh username@192.168.1.50 "mkdir -p ~/.ssh && cat ~/id_ed25519.pub >> ~/.ssh/authorized_keys && rm ~/id_ed25519.pub"

# Verify key login works:
ssh username@192.168.1.50
```

Do not proceed until key login is confirmed working. The next step permanently disables password authentication. If your key is not in place, you will be locked out with no recovery path except physical console access.

Step 3 · Create the SSH hardening drop-in config (Server)

Why use a drop-in file instead of editing sshd_config? The main SSH config file is at `/etc/ssh/sshd_config`. On Bazzite (an immutable OS), OS updates can modify or replace this file. If we edited it directly, our changes could be lost or cause a merge conflict during an update. Files placed in `/etc/ssh/sshd_config.d/` are never touched by OS updates — they are preserved permanently. The `99-hardening.conf` filename sorts last alphabetically, ensuring our settings override any defaults that appear in earlier files.

```
# Write the SSH hardening configuration.
# This heredoc uses no quotes on EOF, so $(whoami) is expanded right now,
# inserting your actual username into the AllowUsers line automatically.
# No manual text editing required.
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf > /dev/null << EOF
Port 22022
PubkeyAuthentication yes
PasswordAuthentication no
PermitRootLogin no
MaxAuthTries 3
LoginGraceTime 20
AllowUsers $(whoami)
EOF
```

Option	What it does and why
`Port 22022`	Moves SSH off the default port 22. Automated bots and scanners constantly probe port 22 worldwide. Moving to a non-standard port eliminates 99%+ of this noise instantly, reducing log pollution and attack surface. This is security through obscurity as a secondary measure — the real security comes from key-only authentication below.
`PubkeyAuthentication yes`	Explicitly enables public key authentication. Without this, a config file loaded later could disable it without warning.
`PasswordAuthentication no`	Permanently disables password login for all accounts. Only your SSH key can authenticate. Makes brute-force attacks mathematically impossible — there is no password to guess.
`PermitRootLogin no`	Blocks direct login as the root user, even with a valid key. An attacker who somehow gets a key can only log in as your regular user, which requires `sudo` to do anything system-level.
`MaxAuthTries 3`	Allows only 3 authentication attempts per connection before the connection is dropped. Limits the window for any creative attack attempt.
`LoginGraceTime 20`	Closes the connection if authentication is not completed within 20 seconds. Reduces the time a half-open connection occupies server resources.
`AllowUsers your_name`	Creates an explicit allowlist — only your account can log in remotely. Even if another user is created on the server, they cannot SSH in. This is defence in depth.

```
# Validate the config syntax before restarting sshd.
# — sshd -t: test/dry-run mode. Parses the config and reports any syntax errors
#   without actually starting or restarting sshd. No output = no errors.
sudo sshd -t

# DO NOT restart sshd yet — we must open port 22022 in the firewall first.
# If we restart sshd now and port 22022 is still blocked, we will be locked out.
```

Step 4 · Configure firewalld and restart sshd in the correct order

What is firewalld? firewalld is the firewall management tool on Fedora/Bazzite. It controls which network traffic is allowed to reach your server. By default, it allows SSH on port 22 and blocks everything else. We need to: (1) allow the new SSH port 22022, (2) restart sshd so it starts listening on 22022, (3) verify we can connect on 22022, and only then (4) close port 22. The order is critical — never close port 22 before confirming port 22022 works, or you will be locked out.

```
# Step 4a — Open port 22022 in the firewall BEFORE restarting sshd.
# — firewall-cmd: the firewalld management tool.
# — --permanent: save this rule permanently. Without this flag, the rule
#   only applies until the next reboot or firewall reload.
# — --add-port=22022/tcp: allow incoming TCP traffic on port 22022.
sudo firewall-cmd --permanent --add-port=22022/tcp

# — --reload: apply all permanent rules to the currently running firewall.
#   Rules marked --permanent don't take effect until a reload.
sudo firewall-cmd --reload

# Step 4b — NOW restart sshd.
# Both prerequisites are now satisfied: SELinux label (Step 1) and firewall rule (above).
# sshd will bind to port 22022 successfully.
sudo systemctl restart sshd
```

Step 4c — TEST NOW before doing anything else. Open a NEW, SEPARATE terminal window on your personal computer and try connecting on the new port:

```
ssh -p 22022 username@192.168.1.50
# — -p 22022: specify the port number (default is 22).
# Expected: successful login.
```

Do not proceed until this succeeds. Keep your existing SSH session open as a safety net while you test.

```
# Step 4d — Remove the old port 22 rule only AFTER confirming port 22022 works.
# — --remove-service=ssh: remove the built-in "ssh" service rule, which
#   allows port 22. We remove the service rule (not a port rule) because
#   firewalld's default zone has an "ssh" service pre-enabled — using
#   --remove-service correctly removes it from the zone's service list.
sudo firewall-cmd --permanent --remove-service=ssh
sudo firewall-cmd --reload

# Verify the final firewall state.
sudo firewall-cmd --list-all
# Expected: ports: 22022/tcp — port 22 must NOT appear.

# Confirm the SELinux label is still in effect:
sudo semanage port -l | grep ssh
# Expected: ssh_port_t    tcp    22022, 22
# (SELinux still shows 22 in its list — this is correct. The firewall
#  now blocks port 22, but SELinux having it listed does no harm.)
```

Step 5 · Configure and enable Fail2ban

> **What is Fail2ban?** Even with key-only authentication, attackers can still flood your server with connection attempts (consuming CPU and filling logs). Fail2ban monitors SSH connection logs in real time. If an IP address makes too many failed authentication attempts within a time window, Fail2ban automatically adds a firewall ban rule blocking that IP entirely. It is a defence-in-depth measure — password authentication is already impossible, but Fail2ban stops noise and protects against edge cases like future misconfigurations.

```
sudo tee /etc/fail2ban/jail.local > /dev/null << 'EOF'
[sshd]
enabled = true
port = 22022
ignoreip = 127.0.0.1/8 ::1 192.168.0.0/16 10.0.0.0/8 172.16.0.0/12
backend = systemd
banaction = firewallcmd-rich-rules
EOF
```

Configuration explained line by line:

`[sshd]` : the jail name — Fail2ban knows how to parse sshd log patterns for this jail type.

`enabled = true` : activate this jail.

`port = 22022` : ban IPs on port 22022, not the default 22.

`ignoreip` : never ban these IP ranges, regardless of attempts. `127.0.0.1/8` = IPv4 localhost, `::1` = IPv6 localhost, `192.168.0.0/16`, `10.0.0.0/8`, and `172.16.0.0/12` = the three RFC 1918 private network ranges. This prevents accidentally banning yourself when accessing from your own LAN, regardless of which private IP range your router uses.

`backend = systemd` : Fedora/Bazzite does not use traditional log files like `/var/log/auth.log`. Instead, everything is logged to the systemd journal (`journalctl`). This tells Fail2ban to read from the journal.

`banaction = firewallcmd-rich-rules` : use firewalld's rich rules to implement bans, which works correctly for both IPv4 and IPv6 on Bazzite.

```
# Enable and start Fail2ban, then verify the SSH jail is active.
sudo systemctl enable --now fail2ban

sudo fail2ban-client status sshd
# Expected: "Status for the jail: sshd" with Port list: 22022
```

PHASE 5 — Docker Initialisation

All remaining steps run on the server via SSH on port 22022.

Step 1 · Enable Docker and add your user to the docker group

```
ssh -p 22022 username@192.168.1.50

# Enable the Docker daemon and start it now.
# Docker is installed from Phase 3 but not yet started.
sudo systemctl enable --now docker

# On Fedora Atomic, the "docker" group may not yet be written to /etc/group
# because the group was part of a layered package, not the base image.
# getent group docker: query the system's group database (NSS) for "docker".
# If it exists but isn't in /etc/group, append it so usermod can find it.
grep -q "^docker:" /etc/group || sudo sh -c "getent group docker >> /etc/group"

# Add your user to the docker group.
# — usermod: modify a user account.
# — -a: append to supplementary groups (without this, -G replaces all groups).
# — -G docker: add the docker group to your supplementary groups.
# — $USER: your current username (the shell expands this automatically).
sudo usermod -aG docker $USER

# Group changes don't take effect until you log out and back in.
# Exit the current SSH session:
exit
# Then reconnect:
ssh -p 22022 username@192.168.1.50
```

Security: the docker group equals effective root. Any user in the docker group can start a container that mounts the entire host filesystem and do anything with it — identical to having passwordless root access. Example: `docker run -v /:/host -it alpine chroot /host` gives a root shell on the host. This is an accepted trade-off on a single-user server where SSH access is locked to your key only. Guard your SSH private key accordingly.

Step 2 · Prevent Docker logs from filling your disk

Why configure log rotation? By default, Docker containers write logs indefinitely to JSON files under `/var/lib/docker/containers/`. A verbose container (like Nextcloud AIO) can generate gigabytes of logs over months, eventually filling the OS drive and causing crashes. We configure a global log rotation policy: each container gets a maximum of 3 log files, each capped at 10 MB — totalling ~30 MB per container regardless of how long it has been running.

```
# Write Docker's daemon configuration file.
# This file configures Docker's global settings — not any specific container.
sudo tee /etc/docker/daemon.json > /dev/null << 'EOF'
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
EOF

# Restart Docker to apply the new daemon configuration.
sudo systemctl restart docker
# Each new container is now capped at ~30 MB of logs total.
# Note: this applies to containers created AFTER this change.
# Pre-existing containers keep their old log settings until recreated.
```

Step 3 · Create the shared Docker network

What is a Docker network? By default, each Docker container is isolated on its own private network. Containers on different networks cannot communicate with each other by name. We create a shared network called `nextcloud-aio` so that Nginx Proxy Manager (NPM) and Nextcloud AIO can talk to each other internally using container names as hostnames — without exposing that communication to your LAN or the internet. Container names on a Docker network work as DNS hostnames: NPM can reach the Nextcloud Apache container simply by typing `nextcloud-aio-apache`.

```
# Create the shared network.
# This command only needs to be run once. Running it again has no effect
# if the network already exists (it just returns an error you can ignore).
docker network create nextcloud-aio
```

PHASE 6 — Data Drive Setup

Complete this before starting any Nextcloud containers. The `NEXTCLOUD_DATADIR` path must exist, be mounted, and have the correct SELinux label before AIO's first start. It cannot be changed afterward without a full data migration.

Ordering is critical. Mount the filesystem FIRST, then apply the SELinux label. Running `restorecon` before mounting labels the OS drive's empty mount-point inode. When the ext4 partition mounts over it, that inode is replaced — the label is lost, and Nextcloud containers fail with "permission denied" errors.

Step 1 · Format, mount, then label the data drive

What are block devices, partitions, and filesystems? A *block device* is a storage device (SSD, HDD) identified in Linux as a file like `/dev/sdb` or `/dev/nvme1n1`. A *partition* is a contiguous region of that device, identified by appending a number: `/dev/sdb1`, `/dev/nvme1n1p1`. A *filesystem* (we use ext4) is a data structure written onto a partition that organises it into files and directories. `lsblk` shows a tree of devices and partitions — use it to identify your data drive before doing anything else.

```
# List all block devices in a tree.
# — lsblk: "list block devices." Shows drives, their partitions, sizes, and
#   mount points. Identify your data drive (the one that is NOT your OS drive).
#   Your OS drive will have a partition mounted at "/" — do not touch that one.
lsblk

# SET THIS VARIABLE — replace with your actual data drive device path.
# NVMe SSD example:  DATA_DRIVE="/dev/nvme1n1"
# SATA/USB SSD/HDD:  DATA_DRIVE="/dev/sdb"
DATA_DRIVE="/dev/nvmeXnY"   # <-- REPLACE THIS

# Double-check you have the right drive before proceeding.
# Verify it does NOT have "/" mounted anywhere in its tree.
echo "Target drive: $DATA_DRIVE" && lsblk "$DATA_DRIVE"

# Create a GPT partition table on the drive.
# WARNING: this irreversibly destroys all existing data on $DATA_DRIVE.
# — parted: the partition management tool.
# — --script: non-interactive mode (no prompts — safe to use in scripts).
# — mklabel gpt: create a GPT (GUID Partition Table) partition table.
#   GPT is the modern standard, supporting drives over 2 TB and up to 128 partitions.
sudo parted "$DATA_DRIVE" --script mklabel gpt

# Create a single partition spanning the entire drive.
# — mkpart primary ext4 1MiB 100%: create one partition from 1 MiB to end of disk.
#   Starting at 1 MiB (not 0) guarantees proper 4K sector alignment on all
#   modern NVMe and SSD hardware — critical for performance.
#   "primary" and "ext4" are labels in parted's legacy terminology; the actual
#   filesystem type is set when we format the partition below.
sudo parted "$DATA_DRIVE" --script mkpart primary ext4 1MiB 100%

# Notify the running kernel that the partition table has changed.
# Without this, the kernel's in-memory partition map is stale and the new
# /dev/... device node for the partition won't exist yet.
sudo partprobe "$DATA_DRIVE"
sleep 2   # Give the kernel 2 seconds to create the new device node.

# Detect the partition name dynamically.
# — lsblk -rn -o NAME: raw output (-r: no tree), no headers (-n), only the NAME column.
# — | tail -n 1: take the last line — the partition we just created.
# — | awk '{print "/dev/"$1}': prepend "/dev/" to get the full device path.
DATA_PART=$(lsblk -rn -o NAME "$DATA_DRIVE" | tail -n 1 | awk '{print "/dev/"$1}')
echo "Partition detected: $DATA_PART"
# Verify: should look like /dev/nvme1n1p1 or /dev/sdb1. If blank, re-check lsblk.

# Format the partition with the ext4 filesystem.
# — mkfs.ext4: creates an ext4 filesystem on the partition, initialising
#   the inode table, journal, and directory structure.
#   After this, the partition can store files.
sudo mkfs.ext4 "$DATA_PART"
```

What is `chattr +i` and why use it here? The `chattr` command changes special file attributes on ext4 filesystems. The `+i` flag sets the "immutable" attribute: not even root can create, delete, rename, or write files inside an immutable directory. We set this on the empty `/srv/nextcloud-data` directory *while the data drive is unmounted*. If the data drive ever fails to mount at boot, `/srv/nextcloud-data` remains as an empty OS-drive directory. Without the immutable flag, Nextcloud AIO (which has `restart: always`) would start, find an empty directory, and either re-initialise a blank database or corrupt the OS drive. With the flag, Docker gets a "permission denied" error and the container safely crashes instead. The kernel automatically overrides the immutable flag on the OS-side directory when the real ext4 partition mounts over it — so this never interferes with normal operation.

IMPORTANT: apply chattr +i BEFORE mounting the drive. If you apply it while the drive is already mounted, you set the immutable flag on the data drive itself, not the OS-side directory. This would prevent Nextcloud from writing to its own live data. Always apply chattr +i while the drive is unmounted, then mount with `mount -a`.

```
# Create the mount point directory.
# — mkdir -p: create the directory and any missing parents.
sudo mkdir -p /srv/nextcloud-data

# Set the immutable flag on the EMPTY directory (drive not yet mounted).
# This is the blind-mount protection — applied while unmounted.
sudo chattr +i /srv/nextcloud-data

# Verify the immutable flag is set.
# — lsattr -d: list attributes of the directory itself (not its contents).
lsattr -d /srv/nextcloud-data
# Expected: ----i-------------- /srv/nextcloud-data
# The "i" in position 5 confirms the immutable flag is set.

# Get the partition's UUID.
# — blkid: queries block device metadata.
# — -s UUID: show only the UUID attribute.
# — -o value: output the value only (no label prefix).
# A UUID is a unique identifier assigned when the filesystem was formatted.
# We use it in /etc/fstab instead of the device path (/dev/sdb1) because
# device paths can change if you add another drive, while UUIDs never change.
PART_UUID=$(sudo blkid -s UUID -o value "$DATA_PART")
echo "Captured UUID: $PART_UUID"
# Expected: a non-empty string like: a1b2c3d4-e5f6-7890-abcd-ef1234567890
# If blank, $DATA_PART is wrong — stop and re-check lsblk.

# Preview the fstab line before writing it:
echo "UUID=${PART_UUID} /srv/nextcloud-data ext4 defaults,nofail 0 2"
# Verify it starts with "UUID=" followed by a real UUID, not "UUID= /srv...".
```

What is /etc/fstab? The fstab file (filesystem table) tells Linux which filesystems to mount at boot, where to mount them, and with what options. Each line has 6 fields separated by spaces:
Field 1: the device — we use `UUID=...` for stability.
Field 2: the mount point — where in the directory tree the device appears (`/srv/nextcloud-data`).
Field 3: filesystem type (`ext4`).
Field 4: mount options. `defaults` = standard options (read/write, auto-mount, etc.). `nofail` = if this device is absent at boot, log a warning and continue booting instead of dropping to emergency mode. Critical for a headless server — without it, a missing drive makes the server unreachable.
Field 5: dump flag (`0` = don't include in the legacy dump backup tool).
Field 6: fsck pass order (`2` = check this filesystem after the root filesystem).

```
# Append the fstab line safely using tee.
# — echo "...": print the fstab line to stdout.
# — | sudo tee -a /etc/fstab: sudo tee reads stdin and appends (-a) to the file.
#   We pipe through tee because /etc/fstab is root-owned and requires sudo.
echo "UUID=${PART_UUID} /srv/nextcloud-data ext4 defaults,nofail 0 2" \
  | sudo tee -a /etc/fstab

# Validate the fstab syntax. This catches typos before they cause boot failures.
# — findmnt --verify: parse fstab and report any errors or inconsistencies.
#   Warnings about options are acceptable. Errors must be fixed before rebooting.
sudo findmnt --verify
# Expected: no errors. Warnings about unknown options are fine.
```

```
# Mount all filesystems listed in fstab that aren't already mounted.
# — mount -a: read /etc/fstab and mount everything listed.
#   This is equivalent to what systemd does at boot.
sudo mount -a

# Verify the data drive is now mounted.
# — df -h: "disk free" — shows mounted filesystems and their usage.
# — -h: human-readable sizes (GB, MB instead of bytes).
df -h /srv/nextcloud-data
# Expected: shows the data drive's size — NOT the OS drive's size.
# If it shows the same size as your OS drive, the mount failed — check journalctl.
```

Why use /var/srv instead of /srv for the SELinux rule? On Fedora Atomic, `/srv` is a symbolic link that points to `/var/srv` . This is part of the immutable OS design — writable data goes under `/var` . SELinux's `semanage fcontext` stores the file context rule literally in a policy database. If we store the rule as `/srv/...` , SELinux labels the *symlink's target* with `var_t` (the default type for `/var` contents) rather than `container_file_t` . Nextcloud containers — which run with the `container_t` domain — cannot write to `var_t` paths. Using the canonical `/var/srv` path ensures the correct label is applied and inherited by all files Nextcloud creates.

```
# Apply the persistent SELinux file context rule using the canonical path.
# — semanage fcontext: manage SELinux file context (label) rules.
# — -a: add a new rule.
# — -t container_file_t: the SELinux type that container processes are
#   permitted to read and write. Nextcloud's Docker containers run in the
#   container_t domain and can only access files labelled container_file_t.
# — "/var/srv/nextcloud-data(/.*)?": this regex applies the label to:
#   - the directory itself (/var/srv/nextcloud-data)
#   - all files and subdirectories inside it (/.*)?
#   We use /var/srv because /srv is a symlink to /var/srv on Fedora Atomic.
sudo semanage fcontext -a -t container_file_t "/var/srv/nextcloud-data(/.*)?"

# Apply the rule to the already-mounted filesystem.
# — restorecon: restore file security contexts to their policy-defined labels.
# — -R: recursive — apply to all files and directories inside.
# — -v: verbose — show each file being relabelled.
# We can use /srv here because the symlink resolves correctly at the filesystem level.
sudo restorecon -Rv /var/srv/nextcloud-data

# Verify the label was applied correctly.
# — ls -Zd: list the SELinux security context (-Z) of the directory itself (-d).
ls -Zd /srv/nextcloud-data
# Expected: system_u:object_r:container_file_t:s0 /srv/nextcloud-data
# The container_file_t label must appear.
```

Why semanage fcontext instead of chcon? `chcon` labels files by directly modifying their extended attributes — but these are reset to default values if the filesystem is ever relabelled (which happens during some Fedora Atomic OS updates). `semanage fcontext` writes the rule into SELinux's persistent policy database. When `restorecon` runs (during OS updates), it reads the policy database and re-applies the correct label — so `container_file_t` is preserved forever. Additionally, any new file created inside the directory automatically inherits `container_file_t` without manual intervention.

PHASE 7 — Deploying Services

DuckDNS · Nginx Proxy Manager · Nextcloud AIO

What is a Docker Compose file? `docker-compose.yml` is a YAML configuration file that describes one or more containers as a single unit. Instead of typing a long `docker run` command with dozens of flags every time, you describe the container's configuration once in this file and manage it with simple commands like `docker compose up -d` and `docker compose down`. Each service in the file becomes a container. YAML is indentation-sensitive — 2 spaces per level. Never use tabs; always use spaces.

What do restart policies mean?
`restart: unless-stopped`: restart the container automatically if it crashes or the Docker daemon restarts — but NOT if you manually stopped it with `docker compose down`.
`restart: always`: always restart, including after a manual stop. Used by Nextcloud AIO's mastercontainer because it manages other containers and must always be running.

Step 1 · DuckDNS auto-updater

Keeps your domain pointed at your home IP. Uses a Docker named volume (managed by Docker) — no bind mounts or SELinux changes needed.

Clock check before proceeding. Containers running in the wrong timezone or with a drifted hardware clock (RTC) cause Nextcloud cron skew, incorrect log timestamps, and calendar event drift. Perform both checks now:

```
# Check your configured timezone:
timedatectl | grep "Time zone"
# Expected: Time zone: Your/Timezone (TZ, +OFFSET)
# Copy the value between the colon and the parenthesis for use below (e.g. Europe/Berlin).

# Check if the hardware clock (RTC) is using local time:
timedatectl | grep "RTC in local TZ"
# Expected: RTC in local TZ: no
# If it shows "yes" — common on systems previously dual-booted with Windows,
# where Windows sets the hardware clock to local time — fix it now:
sudo timedatectl set-local-rtc 0
# Explanation: UTC mode means the hardware clock ticks in UTC (Coordinated Universal
# Time), and Linux applies your timezone offset in software. Local time mode can
# cause clocks to jump by 1-2 hours during DST transitions, causing silent failures
# in TLS certificate validation and Nextcloud session tokens.
```

```
mkdir -p ~/docker/duckdns && cd ~/docker/duckdns

# Capture your exact timezone string for use in the compose file.
# — timedatectl show -p Timezone --value: print only the Timezone property value.
#   This is the exact tz database string (e.g., "America/New_York") needed by containers.
TZ_VALUE=$(timedatectl show -p Timezone --value)
echo "Using timezone: $TZ_VALUE"

# Create the secrets file.
nano .env
```

Paste into `.env`. Keep comments on their own lines — trailing inline comments after a value can be parsed as part of the token string:

```
# subdomain only — not the full domain (just "yourname", not "yourname.duckdns.org"):
DUCKDNS_SUBDOMAINS=yourname
# Your DuckDNS token (no spaces, no quotes):
DUCKDNS_TOKEN=your-token-here
```

```
# chmod 600: set permissions to owner-read/owner-write only.
# 6 = read(4) + write(2) = rw for owner. 0 0 = no permissions for group or others.
# This prevents other users on the system from reading your DuckDNS token.
chmod 600 .env
```

Create docker-compose.yml:

```
cat > docker-compose.yml << EOF
services:
  duckdns:
    image: lscr.io/linuxserver/duckdns:latest
    # image: the Docker image to use. Docker downloads it from the registry
    # on first start. :latest means use the most recent published version.
    container_name: duckdns
    restart: unless-stopped
    labels:
      - "com.centurylinklabs.watchtower.enable=true"
      # Watchtower reads this label — containers with enable=true are
      # automatically updated when a new image is available (Appendix D).
    env_file:
      - .env
      # Load environment variables from .env. The container receives
      # DUCKDNS_SUBDOMAINS and DUCKDNS_TOKEN as environment variables.
    environment:
      - SUBDOMAINS=\${DUCKDNS_SUBDOMAINS}
      - TOKEN=\${DUCKDNS_TOKEN}
      - TZ=${TZ_VALUE}
      # TZ: the container's internal timezone. Affects log timestamps
      # and any time-based logic inside the container.
EOF

cd ~/docker/duckdns && docker compose up -d
# — docker compose up: create and start the containers defined in docker-compose.yml.
# — -d: detached mode — run in the background. Without -d, the container's
#   log output fills your terminal and stops when you close the terminal.
```

Step 2 · Nginx Proxy Manager (NPM)

> **What is a reverse proxy?** A reverse proxy sits in front of your services and routes incoming requests. When a visitor connects to `yourname.duckdns.org`, their request arrives at NPM (on port 443). NPM looks at the domain name, finds the matching proxy host rule, and forwards the request internally to the correct container. The visitor never connects directly to Nextcloud — only NPM is exposed to the internet. NPM also handles TLS termination: it obtains and renews SSL certificates from Let's Encrypt, encrypting all traffic between visitors and your server.

```
mkdir -p ~/docker/npm && cd ~/docker/npm

# Pre-create the bind-mount directories as your user.
# If these don't exist when Docker starts the container, Docker creates them as
# root:root. The NPM container runs as a non-root user internally and cannot
# write to root-owned directories, causing a silent crash loop.
# Creating them as your user first prevents this.
mkdir -p data letsencrypt
sudo chown -R $USER:$USER ./data ./letsencrypt

TZ_VALUE=$(timedatectl show -p Timezone --value)
```

Create docker-compose.yml:

```
cat > docker-compose.yml << EOF
services:
  app:
    image: 'jc21/nginx-proxy-manager:latest'
    container_name: nginx-proxy-manager
    restart: unless-stopped
    labels:
      - "com.centurylinklabs.watchtower.enable=true"
    ports:
      - '80:80'
        # Format: 'host_port:container_port'
        # 80:80 maps the server's port 80 to the container's port 80.
        # Port 80 handles HTTP traffic and Let's Encrypt HTTP challenges.
      - '443:443'
        # Port 443 handles all HTTPS traffic from the internet.
      - '127.0.0.1:81:81'
        # Port 81 is NPM's admin web panel.
        # 127.0.0.1:81 binds it to localhost only — it is NOT accessible
        # from your LAN or the internet, only via SSH tunnel (Phase 10).
    networks:
      - nextcloud-aio
    volumes:
      - ./data:/data:z
        # ./data (on the host) is mounted as /data inside the container.
        # :z is an SELinux volume label — it tells the kernel to relabel
        # the host directory with container_file_t, allowing Docker access.
        # Without :z, SELinux blocks the container from accessing host paths.
      - ./letsencrypt:/etc/letsencrypt:z

networks:
  nextcloud-aio:
    external: true
    # external: true means Docker should use the existing "nextcloud-aio"
    # network we created in Phase 5, rather than creating a new one.
    # This is how NPM and Nextcloud AIO can communicate by container name.
EOF

cd ~/docker/npm && docker compose up -d
```

Step 3 · Nextcloud All-in-One

> **What is Nextcloud AIO?** Nextcloud AIO (All-in-One) is a special "mastercontainer" that manages all the other containers Nextcloud needs: the main app server, a database, a caching layer, a cron job runner, and optionally a full-text search engine and collaboration tools. Instead of manually configuring each component separately, the mastercontainer orchestrates everything. It connects to the Docker socket to create, start, and stop child containers on your behalf. This is why native Docker is required — Podman's Docker compatibility is not sufficient for socket-level orchestration.

> **If you want AMD GPU hardware acceleration for Nextcloud video transcoding:** Do not run `docker compose up -d` below yet. See Appendix C first — the compose file needs modification before the initial container start.

Create docker-compose.yml:

```
mkdir -p ~/docker/nextcloud && cd ~/docker/nextcloud

# Single-quoted 'EOF': no variable expansion inside the heredoc.
# All $ signs are written literally into the file — that is correct here
# because these variables are meant to be read by Docker Compose, not the shell.
cat > docker-compose.yml << 'EOF'
volumes:
  nextcloud_aio_mastercontainer:
    name: nextcloud_aio_mastercontainer
    # This named volume stores AIO's own configuration and state.
    # Giving it an explicit name prevents Docker from auto-generating a long
    # random name that is hard to identify in docker volume ls.

services:
  nextcloud-aio-mastercontainer:
    image: ghcr.io/nextcloud-releases/all-in-one:latest
    init: true
    # init: true runs a lightweight init process (tini) as PID 1 inside the container.
    # This ensures zombie processes are reaped correctly and signals are forwarded
    # properly when Docker stops the container.
    restart: always
    # always: restart even after a manual docker compose down. The mastercontainer
    # must always be running because it manages all child containers.
    container_name: nextcloud-aio-mastercontainer
    security_opt:
      - label:disable
      # REQUIRED. Disables SELinux label confinement for this container.
      # Without this: (1) SELinux blocks access to the Docker socket, preventing
      # the mastercontainer from managing child containers at all, and (2) if you
      # enable GPU hardware acceleration (Appendix C), SELinux blocks access to
      # /dev/dri/renderD128. This is a targeted exemption — the container is still
      # isolated from the host OS via Docker's namespace and cgroup mechanisms.
    labels:
      - "com.centurylinklabs.watchtower.enable=false"
      # Nextcloud AIO must NOT be auto-updated by Watchtower. Updates are done
      # manually through the AIO admin panel to ensure safe database migrations.
    volumes:
      - nextcloud_aio_mastercontainer:/mnt/docker-aio-config
      - /var/run/docker.sock:/var/run/docker.sock:ro
        # /var/run/docker.sock is the Docker API socket — a Unix socket file
        # that the Docker daemon listens on. Mounting it inside the container
        # gives the mastercontainer the ability to communicate with Docker,
        # creating and managing child containers on your behalf.
        # :ro means read-only for the socket FILE ITSELF (the container cannot
        # delete or replace the socket), but bidirectional API communication
        # still flows through it normally.
    ports:
      - '127.0.0.1:8080:8080'
        # The AIO admin/setup panel. Bound to localhost only — not reachable
        # from your LAN. Access via SSH tunnel in Phase 10.
    networks:
      - nextcloud-aio
    environment:
      - APACHE_PORT=11000
        # The internal port where AIO's Apache web server listens.
      - APACHE_IP_BINDING=127.0.0.1
        # Bind Apache to localhost only. This prevents anyone on your LAN from
        # bypassing NPM and connecting directly to port 11000.
        # NPM can still reach it through the shared Docker network by container name.
      - NEXTCLOUD_DATADIR=/srv/nextcloud-data
        # The path where Nextcloud stores all user data, database files, and
        # configuration. This maps to your data drive from Phase 6.
        # This setting cannot be changed after the first start without a full
        # data migration.
      # Uncomment ONLY if your router lacks NAT loopback (see Phase 10 Step 3):
      # - SKIP_DOMAIN_VALIDATION=true

networks:
  nextcloud-aio:
    external: true
EOF

cd ~/docker/nextcloud && docker compose up -d
```

PHASE 8 — Session Management Scripts

start-kde · start-gaming · stop-session

Three scripts control all graphical session transitions, called via SSH from any device. Placed in `/usr/local/bin/` with `root:root` ownership — mandatory for any script granted passwordless `sudo` access.

The core problem this phase solves: When you SSH in and run a command, your SSH connection is a "session" tracked by systemd-logind. Everything you start from that SSH session lives in the same "cgroup" (control group — a Linux kernel mechanism for grouping processes). When you disconnect (close the terminal), logind's `KillUserProcesses=yes` setting sends SIGTERM to everything in that cgroup. Any background process you launched — including a timer meant to restore the headless config — gets killed. `systemd-run` creates a *transient system-level service* that lives outside your SSH session's cgroup, immune to this cleanup. It is owned by systemd itself.

How the SDDM one-shot pattern works: The start scripts work in three phases: (1) write a temporary autologin config pointing to the desired session, (2) restart SDDM — which reads the config synchronously during startup and launches the session, (3) schedule a `systemd-run` transient timer to delete the launch config and restore the headless override 10 seconds later. Since `systemctl restart sddm` is synchronous (it waits for SDDM to start before returning), SDDM has already read and acted on the launch config before the cleanup timer fires. The SSH terminal returns to you immediately.

Stale timer cancellation: Each script begins by cancelling any pending `restore-headless-*` transient timers. If a previous script run was interrupted (e.g., power loss) before the timer fired, a stale timer could fire mid-transition and overwrite the config of the session currently being launched. Cancelling stale timers at the start of each script makes the operation idempotent — safe to run multiple times regardless of previous state.

Step 1 · Security model

We grant passwordless root execution (NOPASSWD) via the sudoers file. Scripts must be owned by root and stored in a path your regular user cannot write to. `/usr/local/bin/` is persistent and mutable on Fedora Atomic, but only root can modify files in it when ownership is `root:root`. If a script in your home directory had NOPASSWD, any process running as you could overwrite it with arbitrary commands — a complete privilege escalation.

Why `$(logname)` instead of `$USER`? `$USER` is an environment variable set when you log in. If you run `sudo -i` or `sudo su`, `$USER` changes to "root". `logname` reads your original login name directly from the kernel's login record — it always returns your actual account name, even from inside a root shell. This makes the username substitution safe regardless of how the command is invoked.

Step 2 · Write start-kde.sh

```
# Write the script file using tee (for root write access) with a single-quoted
# heredoc (no variable expansion — the script is written exactly as shown).
sudo tee /usr/local/bin/start-kde.sh > /dev/null << 'EOF'
#!/bin/bash
# start-kde.sh — Launch a KDE Wayland session on the physical display.
# Refuses if a graphical user session is already active on seat0.
# Location: /usr/local/bin/start-kde.sh (root-owned, not user-writable)
# Usage: ssh -p 22022 user@host 'sudo start-kde.sh'

SDDM_USER="yourusername"       # Replaced automatically by sed below
KDE_SESSION="plasma"           # Matches plasma.desktop in /usr/share/wayland-sessions/
LAUNCH_CONF="/etc/sddm.conf.d/zz-session-launch.conf"
HEADLESS_CONF="/etc/sddm.conf.d/zzz-headless-server.conf"

log() { echo "[start-kde $(date '+%H:%M:%S')] $*"; }

# --- 1. Cancel any pending restore-headless timers ---
# systemctl list-units --type=timer: list all active timer units.
# grep '^restore-headless-': match only our transient restore timers.
# systemctl stop: deactivate each one before it fires.
while IFS= read -r unit; do
    log "Cancelling stale timer: $unit"
    systemctl stop "$unit" 2>/dev/null
done < <(systemctl list-units --type=timer --no-legend 2>/dev/null \
         | awk '{print $1}' | grep '^restore-headless-')

# --- 2. Guard: refuse if a seat0 user session is already active ---
# loginctl list-sessions: list all sessions tracked by systemd-logind.
# For each session ID, we query three properties:
#   Seat: which seat (hardware display) the session is on.
#   State: active = currently focused, online = running but not focused.
#   Class: user = a real user session, greeter = SDDM's login screen.
# We ignore greeter sessions — SDDM's idle login screen is NOT a user session.
ACTIVE=$(loginctl list-sessions --no-legend 2>/dev/null \
  | awk '{print $1}' \
  | while read -r sid; do
      seat=$(loginctl  show-session "$sid" -p Seat  --value 2>/dev/null) || continue
      state=$(loginctl show-session "$sid" -p State --value 2>/dev/null) || continue
      class=$(loginctl show-session "$sid" -p Class --value 2>/dev/null) || continue
      [[ "$class" != "user" ]] && continue
      [[ "$seat" == "seat0" ]] && \
      [[ "$state" == "active" || "$state" == "online" ]] && \
        { echo "$sid"; break; }
    done)

if [[ -n "$ACTIVE" ]]; then
    log "ERROR: A graphical session ($ACTIVE) is already active on seat0."
    log "Log out of the current session first, then re-run this script."
    exit 1
fi
```

```
# --- 3. Write the one-shot autologin config ---
rm -f "$HEADLESS_CONF"       # Remove headless config so it doesn't conflict.
mkdir -p /etc/sddm.conf.d
cat > "$LAUNCH_CONF" << CFGEOF
[Autologin]
User=${SDDM_USER}
Session=${KDE_SESSION}
Relogin=false
CFGEOF
log "SDDM autologin config written for KDE Wayland."

# --- 4. Restart SDDM ---
# systemctl reset-failed sddm: clear any previous failed state so restart works.
# systemctl restart sddm: stop and start SDDM. This is synchronous — it waits
# for SDDM to finish starting before this line returns. SDDM reads LAUNCH_CONF
# during startup, finds a valid User= and Session=, and autologins.
log "Restarting SDDM..."
systemctl reset-failed sddm 2>/dev/null
systemctl restart sddm
log "KDE Wayland session launching on physical display."

# --- 5. Schedule headless restoration as a system-level transient service ---
# systemd-run creates a transient unit managed by systemd itself.
# It runs OUTSIDE the SSH session's cgroup — immune to KillUserProcesses=yes.
# — --no-ask-password: don't prompt for a password.
# — --collect: garbage-collect the unit from systemd's memory after it finishes,
#   preventing accumulation of old unit names if this script runs many times.
# — --unit="restore-headless-$(date +%s)": unique unit name using a timestamp,
#   preventing collisions if the script is called in quick succession.
# — --on-active=10: fire the service 10 seconds after this command runs.
# The bash command: delete the launch config, write the headless config.
systemd-run --no-ask-password --collect \
    --unit="restore-headless-$(date +%s)" \
    --on-active=10 \
    /bin/bash -c 'rm -f /etc/sddm.conf.d/zz-session-launch.conf && \
      printf "[Autologin]\nUser=\nSession=\nRelogin=false\n" \
      > /etc/sddm.conf.d/zzz-headless-server.conf'
log "Headless config restoration scheduled in 10 s."
EOF

# Make the script executable.
# — chmod 755: owner=rwx (read/write/execute), group=r-x, others=r-x.
#   755 allows everyone to read and execute, but only root to modify.
sudo chmod 755 /usr/local/bin/start-kde.sh

# Auto-substitute your username into the script.
# — sed -i: in-place substitution — edit the file directly.
# — "s/yourusername/$(logname)/": replace the placeholder with your real username.
#   $(logname) is evaluated NOW (in this shell) and expands to your login name.
sudo sed -i "s/yourusername/$(logname)/" /usr/local/bin/start-kde.sh

# Verify the substitution worked:
grep "SDDM_USER=" /usr/local/bin/start-kde.sh
# Expected: SDDM_USER="your_actual_username" — not "yourusername"
```

Step 3 · Write start-gaming.sh

> The Gaming Mode session file is `gamescope-session-steam.desktop` in current Bazzite builds. If the file is not found at runtime, the script lists all available session files and exits with a clear, actionable error message — making any future Bazzite rename self-diagnosing.

```
sudo tee /usr/local/bin/start-gaming.sh > /dev/null << 'EOF'
#!/bin/bash
# start-gaming.sh — Launch Gaming Mode on the physical display.
# Refuses if a graphical user session is already active on seat0.
# Location: /usr/local/bin/start-gaming.sh (root-owned, not user-writable)
# Usage: ssh -p 22022 user@host 'sudo start-gaming.sh'

SDDM_USER="yourusername"
GAMING_SESSION="gamescope-session-steam"   # Current session file name in Bazzite
LAUNCH_CONF="/etc/sddm.conf.d/zz-session-launch.conf"
HEADLESS_CONF="/etc/sddm.conf.d/zzz-headless-server.conf"

log() { echo "[start-gaming $(date '+%H:%M:%S')] $*"; }

# --- 1. Cancel any pending restore-headless timers ---
while IFS= read -r unit; do
    log "Cancelling stale timer: $unit"
    systemctl stop "$unit" 2>/dev/null
done < <(systemctl list-units --type=timer --no-legend 2>/dev/null \
         | awk '{print $1}' | grep '^restore-headless-')
```

```
# --- 2. Guard: refuse if a seat0 user session is already active ---
ACTIVE=$(loginctl list-sessions --no-legend 2>/dev/null \
  | awk '{print $1}' \
  | while read -r sid; do
      seat=$(loginctl  show-session "$sid" -p Seat  --value 2>/dev/null) || continue
      state=$(loginctl show-session "$sid" -p State --value 2>/dev/null) || continue
      class=$(loginctl show-session "$sid" -p Class --value 2>/dev/null) || continue
      [[ "$class" != "user" ]] && continue
      [[ "$seat" == "seat0" ]] && \
      [[ "$state" == "active" || "$state" == "online" ]] && \
        { echo "$sid"; break; }
    done)

if [[ -n "$ACTIVE" ]]; then
    log "ERROR: A session ($ACTIVE) is already active on seat0."
    log "Log out using the session's own logout button, then re-run this script."
    exit 1
fi

# --- 3. Verify the Gaming Mode session file exists ---
# SDDM launches sessions by looking up a .desktop file in /usr/share/wayland-sessions/.
# If the file doesn't exist, SDDM falls back to showing the login screen.
SESSION_FILE="/usr/share/wayland-sessions/${GAMING_SESSION}.desktop"
if [[ ! -f "$SESSION_FILE" ]]; then
    log "ERROR: Session file not found: $SESSION_FILE"
    log "Available sessions:"
    ls /usr/share/wayland-sessions/ | sed 's/^/  /'
    log "Update GAMING_SESSION in this script to match the correct filename (without .desktop)."
    exit 1
fi

# --- 4. Write the one-shot autologin config ---
rm -f "$HEADLESS_CONF"
mkdir -p /etc/sddm.conf.d
cat > "$LAUNCH_CONF" << CFGEOF
[Autologin]
User=${SDDM_USER}
Session=${GAMING_SESSION}
Relogin=false
CFGEOF
log "SDDM autologin config written for Gaming Mode."

# --- 5. Restart SDDM ---
log "Restarting SDDM..."
systemctl reset-failed sddm 2>/dev/null
systemctl restart sddm
log "Gaming Mode launching on physical display."

# --- 6. Schedule headless restoration ---
systemd-run --no-ask-password --collect \
    --unit="restore-headless-$(date +%s)" \
    --on-active=10 \
    /bin/bash -c 'rm -f /etc/sddm.conf.d/zz-session-launch.conf && \
      printf "[Autologin]\nUser=\nSession=\nRelogin=false\n" \
      > /etc/sddm.conf.d/zzz-headless-server.conf'
log "Headless config restoration scheduled in 10 s."
EOF

sudo chmod 755 /usr/local/bin/start-gaming.sh
sudo sed -i "s/yourusername/$(logname)/" /usr/local/bin/start-gaming.sh
grep "SDDM_USER=" /usr/local/bin/start-gaming.sh
# Expected: SDDM_USER="your_actual_username"
```

Step 4 · Write stop-session.sh

Emergency fallback — terminates whichever graphical session is active on seat0. For normal use, log out via KDE's own logout button or the Power menu in Steam. This script explicitly restarts SDDM and waits until the SDDM greeter session appears on seat0 before returning. The greeter appearing is hardware-level confirmation that SDDM holds DRM master and the GPU is in a clean, releasable state.

> **What is DRM master?** DRM (Direct Rendering Manager) is the Linux kernel subsystem that controls GPU access. Only one process can hold "DRM master" for a given seat at a time — it has exclusive control of the display output and GPU hardware. When KDE or gamescope is running, it holds DRM master. When it exits, it must release it before another process (like a new SDDM-launched session) can take over. The script polls until SDDM's greeter session appears on seat0, which is the observable confirmation that the GPU handover is complete.

```
sudo tee /usr/local/bin/stop-session.sh > /dev/null << 'EOF'
#!/bin/bash
# stop-session.sh — Terminate the active seat0 session and return to headless idle.
# Normal use: log out via KDE's logout button or Steam's Power menu.
# Location: /usr/local/bin/stop-session.sh (root-owned, not user-writable)
# Usage: ssh -p 22022 user@host 'sudo stop-session.sh'

LAUNCH_CONF="/etc/sddm.conf.d/zz-session-launch.conf"
```

```
HEADLESS_CONF="/etc/sddm.conf.d/zzz-headless-server.conf"

log() { echo "[stop-session $(date '+%H:%M:%S')] $*"; }

# --- 1. Cancel any pending restore-headless timers ---
while IFS= read -r unit; do
    log "Cancelling stale timer: $unit"
    systemctl stop "$unit" 2>/dev/null
done < <(systemctl list-units --type=timer --no-legend 2>/dev/null \
         | awk '{print $1}' | grep '^restore-headless-')

# --- 2. Write the headless config immediately ---
# Do this first — before terminating the session — so that SDDM's next
# startup (triggered by the session ending) reads the headless config.
rm -f "$LAUNCH_CONF"
printf '[Autologin]\nUser=\nSession=\nRelogin=false\n' > "$HEADLESS_CONF"
log "SDDM configs confirmed in headless state."

# --- 3. Find and terminate the active seat0 user session ---
ACTIVE=$(loginctl list-sessions --no-legend 2>/dev/null \
  | awk '{print $1}' \
  | while read -r sid; do
      seat=$(loginctl  show-session "$sid" -p Seat  --value 2>/dev/null) || continue
      state=$(loginctl show-session "$sid" -p State --value 2>/dev/null) || continue
      class=$(loginctl show-session "$sid" -p Class --value 2>/dev/null) || continue
      [[ "$class" != "user" ]] && continue
      [[ "$seat" == "seat0" ]] && \
      [[ "$state" == "active" || "$state" == "online" ]] && \
        { echo "$sid"; break; }
    done)

if [[ -n "$ACTIVE" ]]; then
    log "Terminating session $ACTIVE on seat0..."
    log "WARNING: All unsaved work in the terminated session is lost."
    # loginctl terminate-session: send SIGTERM to all processes in the session,
    # then clean up the session's cgroup and release its resources.
    loginctl terminate-session "$ACTIVE"
    # Brief pause to let the session cgroup begin unwinding before SDDM restart.
    sleep 3
else
    log "No active seat0 user session found. Nothing to terminate."
fi

# --- 4. Restart SDDM to reclaim DRM master ---
# After session termination, the GPU's DRM master lock may still be held
# by lingering compositor processes. Restarting SDDM forces it to reclaim
# the DRM master, putting the GPU in a clean initial state.
log "Restarting SDDM to release GPU/DRM lock..."
systemctl reset-failed sddm 2>/dev/null
systemctl restart sddm

# --- 5. Wait for the SDDM greeter to appear on seat0 ---
# The greeter appearing (Class=greeter on seat0) is the observable signal that
# SDDM successfully holds DRM master — meaning the GPU handover from the
# previous session is complete and the hardware is ready for a new session.
# We poll every 1 second for up to 25 seconds. This makes timing self-regulating
# rather than relying on a fixed sleep that might be too short or too long.
log "Waiting for SDDM greeter to initialize (GPU/DRM settle)..."
GREETER_READY=false
for i in $(seq 1 25); do
    sleep 1
    GREETER=$(loginctl list-sessions --no-legend 2>/dev/null \
      | awk '{print $1}' \
      | while read -r sid; do
          seat=$(loginctl  show-session "$sid" -p Seat  --value 2>/dev/null) || continue
          class=$(loginctl show-session "$sid" -p Class --value 2>/dev/null) || continue
          [[ "$class" == "greeter" && "$seat" == "seat0" ]] && \
            { echo "$sid"; break; }
        done)
    if [[ -n "$GREETER" ]]; then
        log "SDDM greeter ready after ${i}s. GPU/DRM fully released."
        GREETER_READY=true
        # Additional buffer: kwin_wayland (KDE's compositor) releases GPU memory
        # allocations asynchronously after the session exits. This 3-second window
        # ensures gamescope can acquire DRM master cleanly after a KDE session.
        sleep 3
        break
    fi
done

if [[ "$GREETER_READY" != "true" ]]; then
    log "Warning: SDDM greeter not detected within 25s. GPU state uncertain — proceed with caution."
```

```
fi

log "Done. SDDM login screen should be on the physical display."
EOF

sudo chmod 755 /usr/local/bin/stop-session.sh
```

Known display settling limitation: If `start-gaming.sh` is invoked within ~1–2 seconds of `stop-session.sh` completing — before the physical display has fully re-synced its signal — gamescope may fail on the first attempt. Wait for the SDDM login screen to be *physically visible* on the display before running `start-gaming.sh`. This is a display hardware settling behaviour, not a script deficiency.

Why does stop-session.sh not kill your SSH connection? SSH runs as a systemd service and creates sessions on a virtual terminal (TTY) with no assigned seat. `loginctl terminate-session` targets only the seat0 session — the one owning the physical display. Your SSH session has no seat assignment and is completely unaffected.

Step 5 · Configure targeted sudo for the scripts

What is the sudoers file? The `/etc/sudoers` file controls which users can run which commands as root using `sudo`. Normally, `sudo` requires your password. `NOPASSWD:` creates an exception for specific commands, allowing them to run as root without a password prompt — necessary for calling these scripts from an automated SSH command where there is no terminal to type a password into. Files placed in `/etc/sudoers.d/` are automatically included by sudoers, keeping our additions separate from the main file.

```
# Write the sudoers drop-in file non-interactively.
# $USER expands to your username in the shell BEFORE sudo runs.
# Run these commands as your regular user (not from sudo -i).
printf "%s ALL=(ALL) NOPASSWD: /usr/local/bin/start-kde.sh\n" "$USER" \
  | sudo tee /etc/sudoers.d/session-scripts > /dev/null
printf "%s ALL=(ALL) NOPASSWD: /usr/local/bin/start-gaming.sh\n" "$USER" \
  | sudo tee -a /etc/sudoers.d/session-scripts > /dev/null
printf "%s ALL=(ALL) NOPASSWD: /usr/local/bin/stop-session.sh\n" "$USER" \
  | sudo tee -a /etc/sudoers.d/session-scripts > /dev/null

# Set strict permissions on the sudoers file.
# 440 = r--r----- : owner and group can read, nobody can write.
# Sudo refuses to load a sudoers file that is world-readable or writable.
sudo chmod 440 /etc/sudoers.d/session-scripts

# Validate sudoers syntax. Sudo will refuse to load a file with syntax errors,
# which would break all sudo access on the system.
# — visudo -c: check mode (parse only, do not edit).
# — -f FILE: validate this specific file.
sudo visudo -c -f /etc/sudoers.d/session-scripts
# Expected: /etc/sudoers.d/session-scripts: parsed OK

# Confirm the file shows your username (not "root"):
cat /etc/sudoers.d/session-scripts
```

Step 6 · Test the scripts manually

```
# Launch KDE Wayland on the physical display:
sudo start-kde.sh
# Expected: KDE appears on the display. SSH terminal returns promptly.

# While KDE is running, try to launch Gaming Mode (must be refused):
sudo start-gaming.sh
# Expected: ERROR — session already active on seat0.

# Log out of KDE via its logout button (Application Menu → Leave → Log Out).
# Then:
sudo start-gaming.sh
# Expected: Gaming Mode launches.

# Test the emergency stop:
sudo stop-session.sh
# Expected: Script blocks 5–15 seconds polling for the greeter, then returns "Done."

# Confirm both session files exist:
ls /usr/share/wayland-sessions/
# Must include: gamescope-session-steam.desktop and plasma.desktop

# Confirm all scripts are root-owned and executable:
ls -la /usr/local/bin/start-kde.sh \
       /usr/local/bin/start-gaming.sh \
       /usr/local/bin/stop-session.sh
# Expected: -rwxr-xr-x 1 root root ... for all three
```

PHASE 9 — krdp Remote Desktop

What is krdp? krdp is KDE's native Wayland RDP server, integrated into KDE Plasma 6.1+. RDP (Remote Desktop Protocol) is a standard protocol for viewing and interacting with a remote desktop over the network. krdp captures frames directly from the running KDE Wayland compositor via PipeWire's screen capture protocol — meaning the compositor renders to an internal framebuffer regardless of whether a physical display is connected or in DPMS sleep. You get a full interactive desktop remotely, with monitors off. No virtual framebuffer (Xvfb), no X11, no extra packages required.

Security: krdp listens on all interfaces by design. krdp uses `QHostAddress::Any` — there is no configuration option to restrict it to localhost (KDE bug #515737). Port 3389 is reachable on your LAN IP. firewalld blocks it because we never add port 3389 to any allowed rule. **NEVER open port 3389 in firewalld or your router.** Access is exclusively via SSH tunnel.

Step 1 · Enable krdp in KDE System Settings

Requires a running KDE session. Run `sudo start-kde.sh` first if needed.

Step	Action
1.	Open **System Settings** (the gear icon in the application menu)
2.	Navigate to **Remote Desktop**
3.	Toggle **"Enable Remote Desktop"** to ON
4.	Set a **username and password** for RDP login — these can differ from your system login credentials
5.	Leave the port at the default **3389**. Click **Apply**. krdp begins listening immediately.

Step 2 · Verify firewall protection (mandatory after enabling krdp)

```
# Confirm krdp is listening on port 3389.
# — ss: socket statistics (the modern replacement for netstat).
# — -t: show TCP sockets.
# — -l: show listening sockets only (not established connections).
# — -n: show numeric addresses/ports (don't resolve to hostnames/service names).
# — -p: show the process using each socket.
ss -tlnp | grep 3389
# Expected: a line showing port 3389 is listening. This is normal and expected.

# Verify firewalld is NOT allowing 3389.
sudo firewall-cmd --list-all
# Carefully check the "ports:" and "services:" lines.
# 3389 must NOT appear anywhere.

# Final test: attempt to connect from another device on your LAN directly
# to 192.168.1.50:3389 WITHOUT an SSH tunnel — the connection must be refused.
```

Step 3 · Connect via SSH tunnel from your personal computer

What is an SSH tunnel? An SSH tunnel forwards a local port on your computer to a remote port on the server, through the encrypted SSH connection. When you run `ssh -L 13389:127.0.0.1:3389 ...`, any program connecting to `localhost:13389` on your computer has its traffic automatically encrypted and forwarded to port 3389 on the server — as if the connection originated from the server itself (127.0.0.1 = server's own localhost). The RDP traffic never travels unencrypted. Port 3389 on the server never accepts external connections directly.

Windows users: If Remote Desktop Services (TermService) is running on your Windows machine, port 3389 is already bound. The tunnel below uses local port **13389** to avoid this collision on all platforms.

```
# On your PERSONAL COMPUTER — keep this terminal open while using RDP.
# — ssh -N: do not execute a remote command (tunnel only — no shell).
# — -o ServerAliveInterval=60: send a keepalive packet every 60 seconds to prevent
#   idle SSH connections from being silently dropped by routers and firewalls.
# — -L 13389:127.0.0.1:3389: forward local port 13389 to server's localhost:3389.
# — -p 22022: our custom SSH port.
ssh -N -o ServerAliveInterval=60 -L 13389:127.0.0.1:3389 -p 22022 username@your-server-ip

# Then connect your RDP client to: localhost:13389
# Use the krdp credentials set in System Settings (not your Linux login password).
```

RDP Client	Platform	Address
Remote Desktop Connection (built-in)	Windows	localhost:13389
Microsoft Remote Desktop	macOS	localhost:13389
Remmina	Linux	localhost:13389
Microsoft Remote Desktop	Android	Run SSH tunnel in Termux first

krdp only runs while a KDE Wayland session is active. If KDE is not running, RDP connections are refused. Closing the SSH tunnel terminal immediately disconnects the RDP session.

PHASE 10 — Link & Initialise Nextcloud

Step 1 · Access admin panels via SSH tunnel

The admin panels are bound to `127.0.0.1` (server's localhost) — invisible to your LAN. Create SSH tunnels from your personal computer and leave them running while you use the panels.

```
# NPM admin panel — available at http://localhost:8081 in your browser
ssh -N -o ServerAliveInterval=60 -L 8081:127.0.0.1:81 -p 22022 username@192.168.1.50

# Nextcloud AIO setup panel — available at https://localhost:9443 in your browser
ssh -N -o ServerAliveInterval=60 -L 9443:127.0.0.1:8080 -p 22022 username@192.168.1.50

# Run each tunnel in a separate terminal window.
# Leave both open while using the corresponding panel.
```

Step 2 · Configure Nginx Proxy Manager

Start the NPM tunnel, then wait 30–60 seconds. On its very first boot, NPM generates its SQLite database and internal SSL keys. Opening the panel too early returns a **502 Bad Gateway** error — wait and retry. Once ready, open `http://localhost:8081`.

Log in with the default credentials `admin@example.com` / `changeme` and **change both immediately**.

Go to **Hosts → Proxy Hosts → Add Proxy Host** and fill in:

Field	Value	Why
Domain Names	yourname.duckdns.org	The public domain name NPM will respond to
Scheme	http	NPM communicates with the Nextcloud Apache container over plain HTTP internally — the encryption happens at NPM's edge
Forward Hostname / IP	nextcloud-aio-apache	The Docker container name of AIO's Apache server. On the shared `nextcloud-aio` network, container names resolve as DNS hostnames. Use the name, not an IP — container IPs can change.
Forward Port	11000	The port AIO's Apache listens on (set via APACHE_PORT in Phase 7)
Cache Assets	ON	NPM caches static files to reduce load on Nextcloud
Block Common Exploits	ON	Adds basic WAF rules blocking common web attack patterns
Websockets Support	ON	Required for Nextcloud's real-time collaboration features
SSL tab	Request new certificate · Force SSL · HTTP/2 · Agree to ToS → Save	NPM requests a Let's Encrypt certificate for your domain. Force SSL redirects all HTTP to HTTPS.

Expect a 502 error when testing the proxy host in NPM. The `nextcloud-aio-apache` container does not exist until the AIO wizard (Step 4) downloads and starts it. A 502 Bad Gateway at this stage means NPM is configured correctly but Nextcloud's Apache has not started yet. This is expected — not an error.

Required: Advanced tab — large file uploads. After saving the proxy host, click **Edit → Advanced tab** and paste:

```
client_max_body_size 0;
proxy_read_timeout 36000s;
proxy_send_timeout 36000s;
proxy_request_buffering off;
```

`client_max_body_size 0`: removes NPM's default upload size limit. Without this, any file larger than ~1 MB uploaded through the Nextcloud web interface fails with HTTP 413 (Payload Too Large).
`proxy_read/send_timeout 36000s`: allows 10-hour timeouts — needed for large file uploads over slow connections that might otherwise be killed mid-transfer.
`proxy_request_buffering off`: sends the request body directly to the backend without buffering it in NPM first. Required for Nextcloud's chunked upload protocol to work correctly.

Step 3 · Fix local network access (NAT loopback)

What is NAT loopback / hairpinning? When you access `yourname.duckdns.org` from inside your own home network, your DNS lookup returns your public IP address. Your device then tries to connect to that public IP — but the packet never leaves your home network. Many consumer routers cannot handle traffic that would need to "loop back" through the router's own NAT translation. The solution is to make your domain resolve to the server's *local* IP address (192.168.1.50) when accessed from inside the home network. This bypasses the router's NAT entirely.

Solutions — try in order:

Solution 1 — Router local DNS override (recommended): In your router's admin panel, look for "Local DNS", "DNS Entries", or "Static Hosts". Add an entry: `yourname.duckdns.org → 192.168.1.50`. This makes your entire home network resolve the domain to the local IP automatically.

Solution 2 — Pi-hole local DNS: If you use Pi-hole for network-level ad blocking, add a "Local DNS Record" entry for the same mapping in Pi-hole's admin panel.

Solution 3 — SKIP_DOMAIN_VALIDATION (last resort for restricted routers):

```
cd ~/docker/nextcloud
# Edit docker-compose.yml: uncomment the SKIP_DOMAIN_VALIDATION line.
# Then restart the mastercontainer:
docker compose down && docker compose up -d
```

`SKIP_DOMAIN_VALIDATION=true` bypasses AIO's domain reachability check at setup time only. Nextcloud itself remains fully secured by NPM and Let's Encrypt — this setting has no effect on security after setup. Use it only if Solutions 1 and 2 are not possible on your router.

Editing your client's /etc/hosts does NOT solve this. The domain validation request originates from inside the AIO Docker container on the server itself — not from your browser. Host file changes on your PC are irrelevant to that check.

Step 4 · Initialise Nextcloud AIO

Stop the NPM tunnel (Ctrl+C). Start the Nextcloud AIO tunnel. Go to `https://localhost:9443`. Your browser will show a security warning about a self-signed certificate — this is expected (it's the AIO admin panel's own certificate, not your public one). Accept and continue. Copy the setup password shown, enter your domain (`yourname.duckdns.org`), and proceed to the container selection screen.

The wizard offers optional containers. Use this table to decide:

Optional Container	Recommendation	Why
Nextcloud Hub version	Always select the latest (Hub 26+)	Includes the most recent features and security patches
Fulltextsearch	Optional — requires 16 GB+ RAM	Indexes file contents for full-text search. Significant RAM overhead. Skip on 8 GB systems.
HaRP	Recommended	Modern replacement for the older Talk socket proxy. Lower resource usage.
Whiteboard / Imaginary	Optional, low impact	Collaborative whiteboard and image preview generation. Add later if needed.

Click **Start Containers** and wait 5–10 minutes for all containers to download and initialise.

No second drive yet for backups? The AIO wizard shows a Borg backup configuration screen before you can log in to Nextcloud for the first time. If you don't have a dedicated backup drive yet, click **"Setup without a backup solution"** to proceed to Nextcloud now. You can configure Borg later. See Phase 11 for what Borg is and what hardware it requires.

PHASE 11 — Testing & Backups

DNS Test

```
# Query Google's public DNS server (8.8.8.8) directly for your domain.
# This bypasses your router's local DNS override and verifies the external DNS record.
# The result must be your home's public IP address.
# — nslookup domain server: look up "domain" using "server" as the DNS resolver.
nslookup yourname.duckdns.org 8.8.8.8

# If nslookup is not available (it's in the bind-utils package):
# — ping -c 1: send 1 ICMP ping and show the resolved IP address.
ping -c 1 yourname.duckdns.org
```

External Test

Disable Wi-Fi on your phone and visit `https://yourname.duckdns.org` over cellular data. A green padlock and the Nextcloud login screen confirms the full chain works: DuckDNS → your public IP → router port forwarding → NPM → Nextcloud AIO → Let's Encrypt certificate.

Session Management Test

```
ssh -p 22022 username@your-server-ip 'sudo start-kde.sh'
# KDE appears on the physical display. SSH terminal returns promptly.

ssh -p 22022 username@your-server-ip 'sudo start-gaming.sh'
# Must output ERROR — KDE is still active.

# Log out of KDE via its logout button (Application Menu → Leave → Log Out). Then:
ssh -p 22022 username@your-server-ip 'sudo start-gaming.sh'
# Gaming Mode appears on the physical display.

ssh -p 22022 username@your-server-ip 'sudo stop-session.sh'
# Blocks ~5–15 s, then: "Done. SDDM login screen should be on the physical display."
```

System State Validation Checklist

Run these after completing all phases. Every check has a specific expected result.

Group A — OS, SSH & Security Layer

```
# 1. Headless SDDM default is the only SDDM config file present at idle:
ls /etc/sddm.conf.d/
# Expected: zzz-headless-server.conf only.

# 2. No active graphical session at idle:
loginctl list-sessions
# Expected: empty or SSH session only — no seat0 entry.

# 3. SSH only on port 22022 — port 22 completely dead:
ss -tlnp | grep sshd
# Expected: exactly one line showing *:22022. No *:22 entry.

# 4. SELinux SSH port label:
sudo semanage port -l | grep ssh
# Expected: ssh_port_t    tcp    22022, 22

# 5. All suspend targets masked:
systemctl status sleep.target suspend.target hibernate.target \
  hybrid-sleep.target suspend-then-hibernate.target \
  | grep -E 'Loaded:|●'
# Expected: all five lines show "masked".

# 6. Fail2ban active on port 22022:
sudo fail2ban-client status sshd
# Expected: Port list: 22022
```

Group B — Storage, Docker & Services

```
# 7. SELinux label on Nextcloud data directory:
ls -Zd /srv/nextcloud-data
# Expected: system_u:object_r:container_file_t:s0 /srv/nextcloud-data

# 8. fstab has nofail option:
grep nextcloud-data /etc/fstab
# Expected: the line contains "nofail" in the options column.

# 9. Immutable guard active on mount point:
lsattr -d /srv/nextcloud-data
# Expected: ----i-------------- /srv/nextcloud-data
# If the "i" is absent: sudo chattr +i /srv/nextcloud-data (only while unmounted)

# 10. Docker accessible without sudo:
docker info 2>&1 | grep 'Server Version'
# Expected: "Server Version: X.Y.Z" — no "permission denied" error.

# 11. All containers running:
docker ps --format 'table {{.Names}}\t{{.Status}}'
# Expected: duckdns, nginx-proxy-manager, nextcloud-aio-mastercontainer — all "Up".

# 12. Firewall state — only 22022/tcp in firewalld:
sudo firewall-cmd --list-all
# Expected: ports: 22022/tcp only.
# 22, 80, 443, 3389 must be absent. (80/443 are managed by Docker — absent here is correct.)

# 13. NPM admin panel reachable from localhost:
curl -sI http://localhost:81 | head -1
# — curl -sI: send a HEAD request (-I) silently (-s) and show only response headers.
# Expected: HTTP/1.1 302 Found  (NPM's redirect to its login page)
```

Group C — Session Scripts, Integrity & Cgroup Escape

```
# 14. Scripts owned by root and executable:
ls -la /usr/local/bin/start-kde.sh \
       /usr/local/bin/start-gaming.sh \
       /usr/local/bin/stop-session.sh
# Expected: -rwxr-xr-x 1 root root ... for all three.

# 15. SSH root login blocked:
ssh root@127.0.0.1 -p 22022
# Expected: Permission denied (publickey)

# 16. Session guard functional (while a session is active):
sudo start-gaming.sh   # or start-kde.sh if gaming mode is running
# Expected: ERROR: A session ... is already active on seat0.

# 17. Docker CE confirmed as a layered package on the immutable OS:
rpm-ostree status
# Expected: docker-ce appears under "LayeredPackages" in the booted deployment.

# 18. Cgroup escape test — systemd-run survives SSH disconnection:
sudo start-kde.sh && exit          # Launch KDE and immediately disconnect
# Wait 10 seconds, then reconnect:
ssh -p 22022 username@your-server-ip
loginctl list-sessions
# Expected: KDE session on seat0 — it launched and survived the disconnection.
ls /etc/sddm.conf.d/
# Expected: zzz-headless-server.conf — the transient timer ran after disconnection.
```

Backups

What is Borg Backup — explained from scratch:
Borg is free, open-source backup software. Your data *never* leaves your home — no cloud accounts, no subscriptions, $0 cost.

How data flows on this server:

- **Live data** → your SSD at `/srv/nextcloud-data`. This is where your files actually live and are served from. When you upload a photo to Nextcloud, it goes here.
- **Borg snapshots** → a second physical drive (USB hard drive, SATA drive, NAS). Borg creates compressed, deduplicated snapshots of your live SSD data at scheduled intervals. If the SSD fails or you accidentally delete something, you restore from the Borg backup.

Hardware required: A second physical drive dedicated to backups. Without it, Borg has nowhere to write. If you don't have one yet, click "Setup without a backup solution" in the AIO wizard and come back to this once you do.

Verify your live storage is healthy: `df -h /srv/nextcloud-data`

Inside the AIO setup panel (`https://localhost:9443`): find the **Borg Backup** section. Set a destination path on your second drive (e.g., `/mnt/backup-drive`) and schedule daily backups.

Perform a test restore immediately after the first backup completes. A backup you have never tested cannot be trusted. The restore path is documented in the AIO panel. Perform a test restore once a month.

APPENDIX A — Troubleshooting

Machine boots to Gaming Mode after initial install

```
# SSH is not yet available on a fresh install.
# Go to Gaming Mode → Power menu → Switch to Desktop.
# Then run Phase 1 Steps 2–4 to enable SSH and apply the headless override.
```

start-kde.sh / start-gaming.sh reports "session already active" but nothing is visible

```
# A session from a previous run is stuck in logind.
loginctl list-sessions
# Note the stuck session ID (e.g., "3" or "c1").
loginctl terminate-session SESSION_ID   # replace with actual ID
sleep 2
sudo start-kde.sh   # or: sudo start-gaming.sh
```

SDDM shows login screen instead of autologging in

```
# The Session= value must exactly match the .desktop filename without the extension.
ls /usr/share/wayland-sessions/
# Expected: plasma.desktop and gamescope-session-steam.desktop

grep 'KDE_SESSION=' /usr/local/bin/start-kde.sh
grep 'GAMING_SESSION=' /usr/local/bin/start-gaming.sh

# Investigate SDDM's log if session names look correct:
journalctl -u sddm -n 50 --no-pager
```

krdp RDP connection refused

```
# 1. Confirm KDE is running:
loginctl list-sessions   # must show a seat0 session

# 2. Confirm krdp is listening:
ss -tlnp | grep 3389   # must show port 3389 listening

# 3. If nothing: re-check Remote Desktop is enabled in KDE System Settings.

# 4. Confirm the SSH tunnel is still open on your personal computer:
#    The "ssh -N -L 13389:..." terminal must still be running.
#    Closing that terminal immediately kills the RDP connection.
```

Docker containers not running after a reboot

```
# Check Docker's service status:
sudo systemctl status docker

# If Docker is not enabled:
sudo systemctl enable --now docker

# Restart each service stack from its directory:
cd ~/docker/npm      && docker compose up -d
cd ~/docker/nextcloud && docker compose up -d
cd ~/docker/duckdns  && docker compose up -d

# Confirm all containers are running:
docker ps --format 'table {{.Names}}\t{{.Status}}'
```

zz-session-launch.conf persists (power loss during a script run)

```
# The background restoration timer was interrupted before it could run.
# Manual fix — restore the headless state:
sudo rm -f /etc/sddm.conf.d/zz-session-launch.conf
sudo tee /etc/sddm.conf.d/zzz-headless-server.conf > /dev/null << 'EOF'
[Autologin]
User=
Session=
Relogin=false
EOF
ls /etc/sddm.conf.d/
# Expected: only zzz-headless-server.conf
```

Data drive not mounting — system dropped to emergency mode

This should not happen if `nofail` is in fstab (Phase 6). On an older setup without `nofail`:

```
# Connect a display and keyboard.
# At the emergency shell, edit fstab:
nano /etc/fstab
# Find the /srv/nextcloud-data line and add "nofail" to the options column.
# Then reboot:
systemctl reboot
```

APPENDIX B — Normal Workflows

Quick Reference — commands to run from your personal computer

Use KDE on the TV or monitor

```
ssh -p 22022 username@your-server-ip 'sudo start-kde.sh'
# KDE Wayland session appears on the physical display.
# To stop: Application Menu → Leave → Log Out.
```

Work remotely via RDP

```
# Step 1 — Start KDE:
ssh -p 22022 username@your-server-ip 'sudo start-kde.sh'

# Step 2 — Open the SSH tunnel (keep this terminal open):
ssh -N -o ServerAliveInterval=60 -L 13389:127.0.0.1:3389 -p 22022 username@your-server-ip

# Step 3 — Connect your RDP client to: localhost:13389
# Use the krdp credentials set in KDE System Settings.
```

Game

```
# If KDE is running, log out first via KDE's logout button. Then:
ssh -p 22022 username@your-server-ip 'sudo start-gaming.sh'
# To stop: Power menu in Steam Big Picture → Exit to Desktop.
# (This returns to the idle SDDM login screen.)
```

Force-stop a stuck session

```
ssh -p 22022 username@your-server-ip 'sudo stop-session.sh'
# WARNING: all unsaved work in the terminated session is lost.
# Script blocks ~5–15 s polling for the SDDM greeter, then returns.
```

Check what is currently running

```
loginctl list-sessions
# seat0 entry present = a graphical session is active on the physical display.

loginctl show-session -p Type -p Desktop
# Type=wayland, Desktop=KDE → KDE Plasma is running.
# Type=wayland, Desktop=gamescope → Gaming Mode is running.

ls /etc/sddm.conf.d/
# Healthy server state: only zzz-headless-server.conf is present.
```

Never run `firewall-cmd --reload` while Docker is running. `firewalld --reload` flushes and rebuilds the entire nftables ruleset from its stored rules. Docker injects port-forwarding rules directly into nftables at runtime — these are not stored in firewalld's config and are wiped on reload. Consequence: ports 80 and 443 become unreachable, taking Nextcloud offline with no error message. If you must reload firewalld (e.g., to apply a new firewall rule), immediately run:

```
sudo systemctl restart docker
# This re-injects Docker's nftables rules and restores external access.
```

Warning: restarting Docker terminates all running containers. If Nextcloud is actively syncing, put it in maintenance mode first:

```
docker exec nextcloud-aio-nextcloud php occ maintenance:mode --on
# ...restart Docker, wait for containers to recover, then:
docker exec nextcloud-aio-nextcloud php occ maintenance:mode --off
```

APPENDIX C — Optional: AMD GPU Hardware Acceleration

For Nextcloud video transcoding · AMD and Intel GPUs only · Complete before first docker compose up -d in Phase 7

AMD and Intel GPUs only. This section configures VA-API GPU hardware acceleration for Nextcloud's preview and video transcoding features. It is tested and supported on AMD and Intel integrated/discrete GPUs. **Do not follow this appendix on NVIDIA systems** — NVIDIA uses a different driver model and render node path. NVIDIA hardware acceleration for Nextcloud requires separate research and is outside the scope of this guide.

Timing requirement. This configuration must be applied *before* running `docker compose up -d` for the first time in Phase 7 Step 3. The AIO mastercontainer runs initialisation scripts on its very first start that configure user permissions for GPU access — this process requires write access to the data directory. If you have already started the containers and want to add hardware acceleration, you must stop all containers (`docker compose down`), remove the AIO data volume (`docker volume rm nextcloud_aio_mastercontainer`), and start fresh. This does not delete your Nextcloud user data, but does reset the AIO configuration — you will need to reconfigure NPM and re-run the AIO wizard. For this reason, if you want GPU acceleration, add it now before Phase 7's `docker compose up -d`.

What is VA-API? VA-API (Video Acceleration API) is a Linux interface for hardware-accelerated video encode and decode operations. Instead of the CPU processing every video frame in software (slow, high CPU usage), VA-API lets applications offload that work to the GPU's dedicated hardware encoder/decoder. For Nextcloud, this speeds up video thumbnail generation and transcoding when previewing video files. For AMD GPUs, VA-API is provided by the Mesa radeonsi driver, which is included in Bazzite.

What is /dev/dri? The DRI (Direct Rendering Infrastructure) subsystem exposes the GPU to userspace applications as device files in `/dev/dri/`. There are two types: `cardX` files (used by compositors for display output) and `renderDX` files (used by applications for GPU compute and hardware transcoding without needing display control). Nextcloud needs access to `renderD128` (or the equivalent on your system) for VA-API transcode operations. The `render` group owns this file — AIO's initialisation will add the container's `www-data` user to this group during first start.

Step 1 · Verify your AMD GPU and render nodes are present

```
# List all GPU device files:
ls -la /dev/dri/
# Expected output example:
# crw-rw----+ 1 root video  226,   0 ... card0
# crw-rw----+ 1 root render 226, 128 ... renderD128
#
# card0: the display output device (used by the compositor).
# renderD128: the render node — this is what Nextcloud needs for VA-API.
# The "render" group owns renderD128.
# If renderD128 is absent, your GPU driver is not loaded correctly — check journalctl.

# Verify VA-API is working on the host (optional, requires libva-utils):
vainfo 2>/dev/null | head -20
# Expected: lists supported VA-API profiles (H.264, HEVC, AV1, etc.)
# If vainfo is not installed: sudo rpm-ostree install libva-utils && systemctl reboot
```

Step 2 · Remove the immutable guard (BEFORE first container start)

```
# The AIO mastercontainer's first-start initialisation writes ownership data
# to the data directory. The chattr +i immutable flag blocks this.
# Remove the flag temporarily so AIO can complete its initialisation.
# IMPORTANT: only do this if the data drive is currently mounted.
df -h /srv/nextcloud-data   # Confirm the data drive is mounted first.

# Remove the immutable flag:
# — chattr -i: remove (-) the immutable (i) attribute.
sudo chattr -i /srv/nextcloud-data

# Verify the flag is gone (the "i" must be absent):
lsattr -d /srv/nextcloud-data
# Expected: ------------------ /srv/nextcloud-data  (no "i" in any position)
```

Step 3 · Update the Nextcloud AIO compose file

```
cd ~/docker/nextcloud

# Add the GPU device mapping and enable the DRI environment variable.
# We append the devices block and add one environment variable to the
# existing docker-compose.yml.
# Replace the environment section in your docker-compose.yml to add the
# NEXTCLOUD_ENABLE_DRI_DEVICE line, and add a devices section.
# The simplest method is to rewrite the file completely:

cat > docker-compose.yml << 'EOF'
volumes:
  nextcloud_aio_mastercontainer:
    name: nextcloud_aio_mastercontainer

services:
  nextcloud-aio-mastercontainer:
    image: ghcr.io/nextcloud-releases/all-in-one:latest
    init: true
    restart: always
    container_name: nextcloud-aio-mastercontainer
    security_opt:
      - label:disable
    labels:
      - "com.centurylinklabs.watchtower.enable=false"
    devices:
      - /dev/dri:/dev/dri
      # Mount the host's /dev/dri directory into the container at the same path.
      # This gives the container access to the GPU render node (renderD128),
      # enabling VA-API hardware transcoding inside the Nextcloud app container.
      # The container's /dev/dri/renderD128 maps directly to the host GPU.
    volumes:
      - nextcloud_aio_mastercontainer:/mnt/docker-aio-config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    ports:
      - '127.0.0.1:8080:8080'
    networks:
      - nextcloud-aio
    environment:
      - APACHE_PORT=11000
      - APACHE_IP_BINDING=127.0.0.1
      - NEXTCLOUD_DATADIR=/srv/nextcloud-data
      - NEXTCLOUD_ENABLE_DRI_DEVICE=true
        # Tells the AIO mastercontainer to: (1) detect the GPU render node,
        # (2) add the www-data user inside the Nextcloud app container to the
        # "render" group so it can access renderD128, and (3) pass the device
        # through to child containers. Without this variable, the devices:
        # mapping above has no effect.
      # Uncomment if needed:
      # - SKIP_DOMAIN_VALIDATION=true

networks:
  nextcloud-aio:
    external: true
EOF
```

Step 4 · Start containers and wait for initialisation

```
# Start the mastercontainer.
cd ~/docker/nextcloud && docker compose up -d

# Monitor container startup. The mastercontainer will pull all child container
# images (this can take 5–15 minutes on the first run).
# — watch: run a command repeatedly every 2 seconds.
# — docker ps: show running containers.
# Ctrl+C to stop watching once all containers show "healthy".
watch docker ps --format 'table {{.Names}}\t{{.Status}}'

# Wait until all AIO-managed containers show "(healthy)" in their status.
# This confirms the initialisation scripts have completed successfully,
# including the GPU group membership setup for www-data.
```

Step 5 · Re-apply the immutable guard

> **Do not skip this step.** Once all containers show healthy, immediately re-apply the immutable flag. This restores blind-mount protection — if the data drive fails to mount on a future reboot, Docker cannot corrupt the OS drive.

```
# Re-apply the immutable flag on the data directory:
sudo chattr +i /srv/nextcloud-data

# Verify:
lsattr -d /srv/nextcloud-data
# Expected: ----i-------------- /srv/nextcloud-data   (the "i" is back)
```

Step 6 · Verify hardware acceleration is active inside Nextcloud

```
# Check that the Nextcloud app container can see VA-API hardware accelerators.
# — docker exec: run a command inside a running container.
# — nextcloud-aio-nextcloud: the name of the Nextcloud app container.
# — ffmpeg -hwaccels: list all available hardware acceleration methods.
docker exec nextcloud-aio-nextcloud ffmpeg -hwaccels 2>/dev/null | grep -i vaapi
# Expected: "vaapi" listed as an available hardware accelerator.
# If vaapi is absent, check that /dev/dri/renderD128 exists on the host
# and that the containers were restarted after adding the devices section.
```

After verifying GPU acceleration works, proceed to Phase 10 to configure NPM and initialise Nextcloud through the AIO wizard. The GPU acceleration works automatically for video previews and transcoding — no additional Nextcloud configuration is required.

APPENDIX D — OS Updates & Rollbacks

Bazzite stages OS updates silently in the background. Applying them requires a reboot — during which your server is offline for 1–2 minutes. All containers come back up automatically (`restart: always` / `restart: unless-stopped`).

```
# If any update causes a problem, roll back to the previous OS image instantly.
# Bazzite always keeps your last two deployments — rollback is always possible.
rpm-ostree rollback && systemctl reboot

# List all available stable builds by tag (useful for targeted rollbacks):
# — skopeo list-tags: list all available image tags in a container registry.
# The Bazzite OS itself is distributed as an OCI container image.
skopeo list-tags docker://ghcr.io/ublue-os/bazzite-deck \
  | grep -- 'stable-' | sort -rV

# Rebase to a specific build (replace YEARMONTHDAY with the target date):
rpm-ostree rebase ostree-image-signed:docker://ghcr.io/ublue-os/bazzite-deck:stable-YEARMONTHDAY
```

After a major OS rebase (e.g., Fedora 40 → 41): Because Docker CE is layered via rpm-ostree, a major version rebase may need the Docker layer reinstalled. If Docker fails to start after a rebase:

```
rpm-ostree rollback && systemctl reboot
```

Then re-follow Phase 3 to reinstall the Docker layer on the new base image.

Watchtower — Auto-update containers

What is Watchtower? Watchtower is a Docker container that watches your other containers for new image versions. When a newer version of an image is published, Watchtower pulls it and restarts the container — like apt upgrade, but for Docker containers. We use opt-in mode: only containers explicitly labelled with `com.centurylinklabs.watchtower.enable=true` are updated (DuckDNS and NPM). Nextcloud AIO containers are excluded from auto-updates because Nextcloud database migrations must be handled through the AIO panel, not by simply restarting with a new image.

```
mkdir -p ~/docker/watchtower && cd ~/docker/watchtower

TZ_VALUE=$(timedatectl show -p Timezone --value)

cat > docker-compose.yml << EOF
services:
  watchtower:
    image: containrrr/watchtower
    restart: unless-stopped
    security_opt:
      - label:disable   # Required for Docker socket access on SELinux systems.
    environment:
      - TZ=${TZ_VALUE}
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    command: --label-enable --cleanup --schedule "0 0 4 * * *"
    # — --label-enable: only update containers with watchtower.enable=true label.
    # — --cleanup: delete the old image after updating to free disk space.
    # — --schedule "0 0 4 * * *": cron format — run at 04:00 every day.
    #   Format: second minute hour day-of-month month day-of-week.
EOF

cd ~/docker/watchtower && docker compose up -d
```